

✓ Problema 1

Un profesor intenta mostrar a sus estudiantes la importancia de los exámenes cortos, aun cuando el 90% de la calificación final esté determinada por los exámenes parciales. Él cree que cuanto más altas sean las calificaciones de los exámenes cortos, más alta será la calificación final. Seleccionó una muestra aleatoria de 15 estudiantes de su clase con los siguientes datos:

Promedio de exámenes cortos	Promedio final
59	64
92	84
72	77
90	80
95	77
87	81
89	80
77	84
76	80
65	69
97	83
42	40
94	78
62	65
91	90

```
import pandas as pd
df=pd.DataFrame({
    'Promedio_exámenes_cortos':[59,92,72,90,95,87,89,77,76,65,97,42,94,62,91],
    'Promedio_final':[64,84,77,80,77,81,80,84,80,69,83,40,78,65,90]
})
df
```

	Promedio_exámenes_cortos	Promedio_final	
0	59	64	
1	92	84	
2	72	77	
3	90	80	
4	95	77	
5	87	81	
6	89	80	
7	77	84	
8	76	80	
9	65	69	
10	97	83	
11	42	40	
12	94	78	
13	62	65	
14	91	90	

Próximos pasos:

[Generar código con df](#)

[New interactive sheet](#)

1.Establesca una variable dependiente (Y) y una variable independiente (X).

```
Y =df['Promedio_final']
X =df['Promedio_exámenes_cortos']
```

2.Realice un diagrama de dispersión para estos datos.

```
# Importamos la librería matplotlib.pyplot y le damos el pseudónimo plt
import matplotlib.pyplot as plt

# Configuración general del gráfico
plt.figure(
    figsize=(6, 4),    # tamaño de la figura (ancho, alto) en pulgadas
    dpi=120             # resolución del gráfico
)
```

```

# Gráfico de dispersión
plt.scatter(
    X, Y,
    marker="*",          # forma: googlear "matplotlib.markers"
    color='#C979B7',     # color de los puntos
    edgecolor='#C979B7', # borde de los puntos
    alpha=0.8,           # transparencia: 0 es invisible y 1 es color s
    s=50,                # tamaño de los puntos
)

# Etiquetas de los ejes
# eje x
plt.xlabel(
    'Temperatura (°C)', # etiqueta del eje x
    fontsize=12 # tamaño de fuente
)

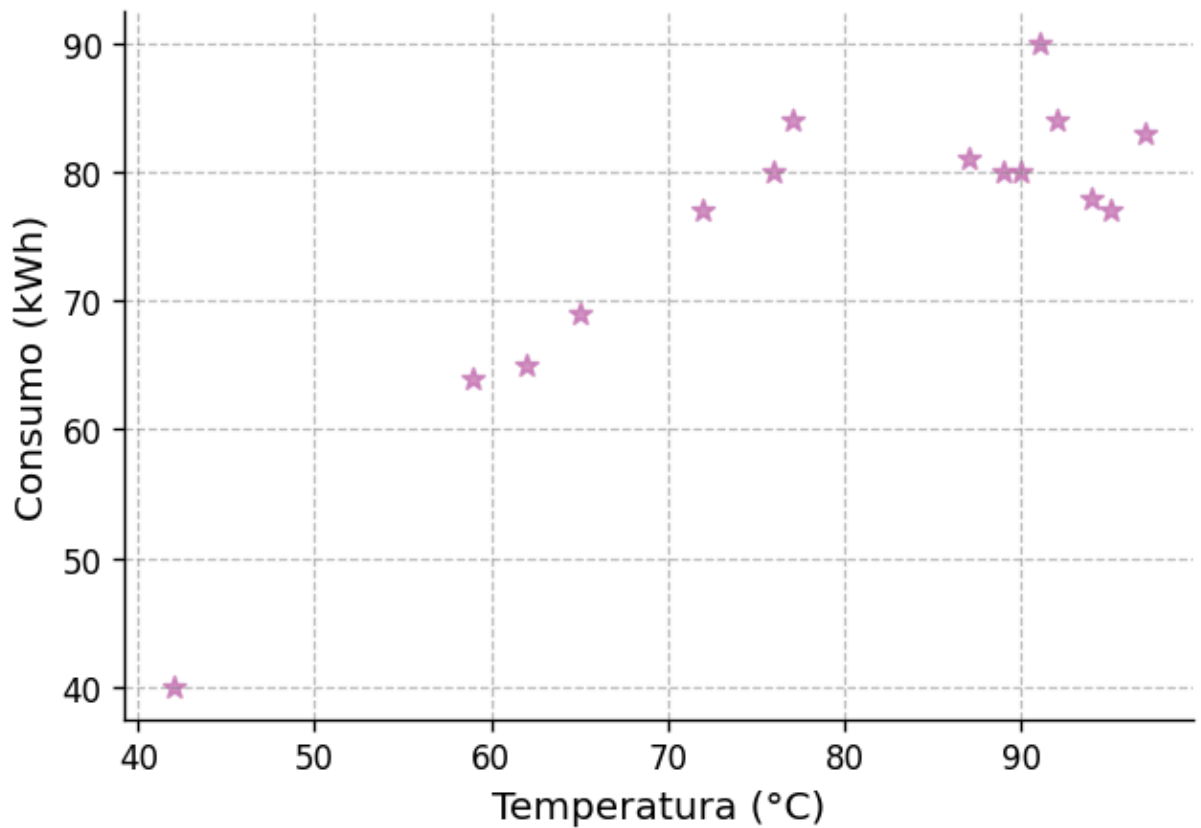
plt.ylabel(
    'Consumo (kWh)', # etiqueta del eje y
    fontsize=12 # tamaño de fuente
)

# Fuente de los ticks
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)

# Márgenes
plt.gca().spines['right'].set_visible(False) # derecha
plt.gca().spines['top'].set_visible(False)   # superior

# Cuadrícula (opcional, pero didáctica)
plt.grid(
    visible=True,
    linestyle='--',
    linewidth=0.7,
    alpha=0.5,
    color="gray"
)

```



- ✓ 4. Calcule el coeficiente de correlación e interprete el resultado.

```
# Importar la función pearsonr desde scipy.stats
from scipy.stats import pearsonr

# Test de Pearson
# H0: rho = 0      (No hay correlación)
# H1: rho ≠ 0     (Sí hay correlación)
# alpha = 0.05

r, valor_p = pearsonr(X,Y)

print(f"Coeficiente de correlación: {r: 0.4f}")
print(f'valor_p: {valor_p: 0.4f}')
```

```
Coeficiente de correlación:  0.8646
valor_p:  0.0000
```

- ✓ 5. Calcule el coeficiente de determinación e interprete el resultado.

```
import statsmodels.api as sm
x_constante = sm.add_constant(X)
```

```
modelo = sm.OLS(Y, x_constante).fit()
y_calculada = modelo.predict(x_constante)
```

```
# Coeficiente de determinación
from sklearn.metrics import r2_score

r2 = r2_score(Y, y_calculada)

print(f"Coeficiente de determinación: {r2: 0.2%}")
```

Coeficiente de determinación: 74.75%

6. Obtenga la recta de regresión ajustada y gráfiquelo sobre el gráfico de dispersión

```
import statsmodels.api as sm
x_constante = sm.add_constant(X)
modelo = sm.OLS(Y, x_constante).fit()
y_calculada = modelo.predict(x_constante)

modelo.params
```

	0
const	24.526822
Promedio_exámenes_cortos	0.643180

dtype: float64

El modelo es:

$$\hat{y} = 24.5268 + 0.643180x$$

donde x es el promedio en exámenes cortos

```
# Importamos la librería matplotlib.pyplot y le damos el pseudónimo plt
import matplotlib.pyplot as plt

# Configuración general del gráfico
plt.figure(
    figsize=(6, 4),  # tamaño de la figura (ancho, alto) en pulgadas
    dpi=120          # resolución del gráfico
)

# Gráfico de dispersión
plt.scatter(
```

```

X, Y,
marker="*",          # forma: googlear "matplotlib.markers"
color='#E38DBF',      # color de los puntos
edgecolor='#E38DBF',  # borde de los puntos
alpha=0.9,            # transparencia: 0 es invisible y 1 es color sólido
s=50,                # tamaño de los puntos
)

plt.plot(
    X, y_calculada,
    color="#77D166",    # color de la línea de regresión
    alpha=0.9,          # transparencia: 0 es invisible y 1 es color sólido
    label="Recta de regresión" # Etiqueta para la leyenda
)

# Las líneas siguientes se usan típicamente en gráficos de residuales, n
# Si quieres ver la línea de regresión con la etiqueta, usa plt.legend()
# plt.axhline(
#     y=0,          # Donde está la línea horizontal
#     label="Recta de regresión", # Etiqueta
#     linestyle = "--", # Estilo de línea
#     color = "black", # color
#     linewidth=1.0   # ancho de línea
# )

# Etiquetas de los ejes
plt.xlabel(
    'Promedio exámenes cortos', # etiqueta del eje x
    fontsize=12 # tamaño de fuente
)

plt.ylabel(
    'Promedio final', # etiqueta del eje y (cambiado de 'Residuales' y '
    fontsize=12 # tamaño de fuente
)

# Fuente de los ticks
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)

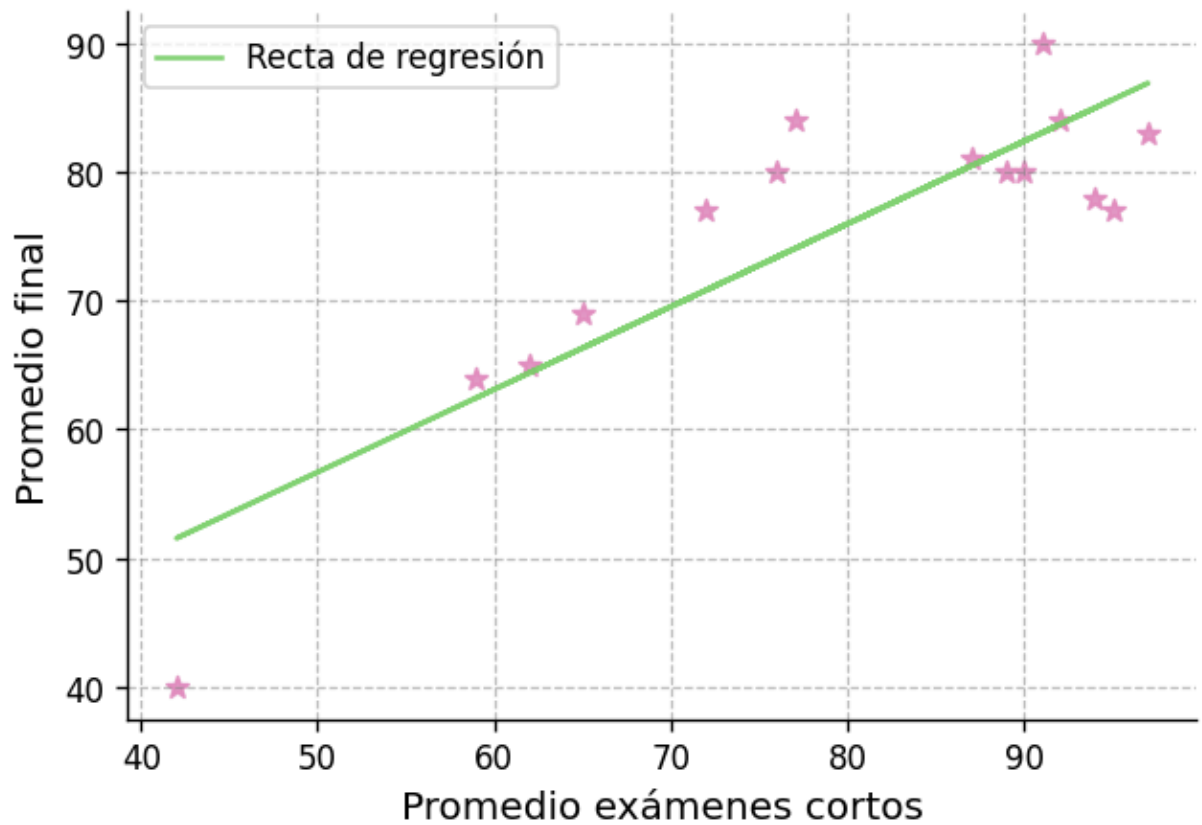
# Márgenes
plt.gca().spines['right'].set_visible(False) # derecha
plt.gca().spines['top'].set_visible(False)   # superior

# Cuadrícula (opcional, pero didáctica)
plt.grid(
    visible=True,
    linestyle='--',
    linewidth=0.7,
    alpha=0.5,
    color="gray"
)

```

)

```
plt.legend() # Muestra la leyenda con la etiqueta de la recta de regresión  
plt.show() # Muestra el gráfico
```



7. Obtenga un intervalo de confianza del 95% para la pendiente de la recta de regresión ajustada (b_1)

```
#intervalos de confianza  
modelo.conf_int(alpha=0.05)
```

	0	1
const	6.441997	42.611646
Promedio_exámenes_cortos	0.419219	0.867141

8. Calcule los residuales y trace un nuevo gráfico de dispersión. Comente, ¿Parece que se verifican los supuestos?

```
residuales = modelo.resid
```

```

# Importamos la librería matplotlib.pyplot y le damos el pseudónimo plt
import matplotlib.pyplot as plt

# Configuración general del gráfico
plt.figure(
    figsize=(6, 4),    # tamaño de la figura (ancho, alto) en pulgadas
    dpi=120             # resolución del gráfico
)

# Gráfico de dispersión de residuales vs. Promedio_exámenes_cortos
plt.scatter(
    X, residuales,
    marker="o",         # forma: googlear "matplotlib.markers"
    color='#C979B7',    # color de los puntos
    edgecolor='#C979B7', # borde de los puntos
    alpha=0.8,          # transparencia: 0 es invisible y 1 es color s
    s=50,               # tamaño de los puntos
)

# Línea horizontal en y=0 para referencia
plt.axhline(
    y=0,               # Donde está la línea horizontal
    color='gray',      # color
    linestyle = '--',  # Estilo de línea
    linewidth=1.5      # ancho de línea
)

# Etiquetas de los ejes
plt.xlabel(
    'Promedio exámenes cortos', # etiqueta del eje x
    fontsize=12 # tamaño de fuente
)

plt.ylabel(
    'Residuales', # etiqueta del eje y
    fontsize=12 # tamaño de fuente
)

# Fuente de los ticks
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)

# Márgenes
plt.gca().spines['right'].set_visible(False) # derecha
plt.gca().spines['top'].set_visible(False)   # superior

# Cuadrícula (opcional, pero didáctica)
plt.grid(
    visible=True,
    linestyle='--',

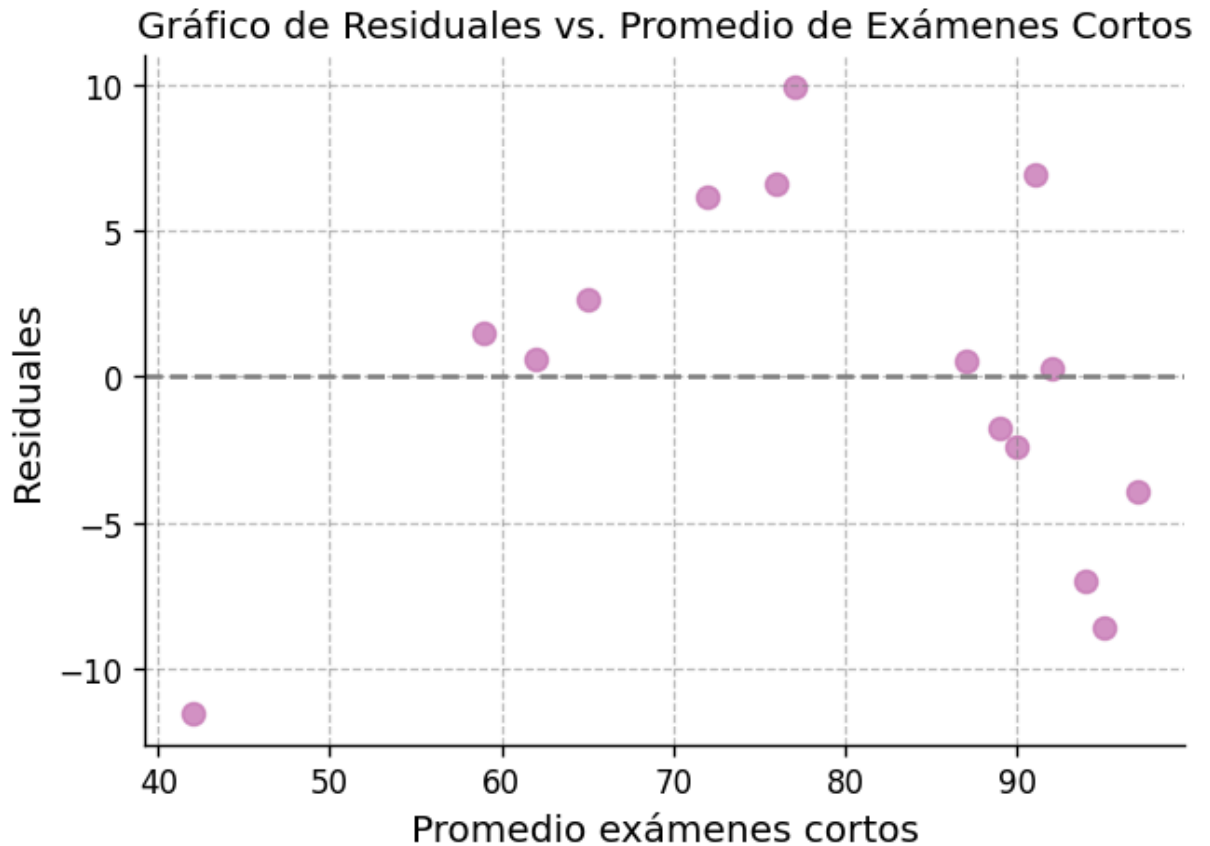
```

```

linewidth=0.7,
alpha=0.5,
color="gray"
)

plt.title('Gráfico de Residuales vs. Promedio de Exámenes Cortos')
plt.show()

```



9. Realice la prueba de Shapiro para los resultados y comente el resultado

```

from scipy.stats import shapiro
import matplotlib.pyplot as plt
import seaborn as sns

#test de shapiro
#H0: Hay nornormalidad
#H1: No hay normalidad

# Prueba de Shapiro-Wilk
stat, valor_p_sh = shapiro(residuales)
print(f"valor-p (Shapiro) = {valor_p_sh}")

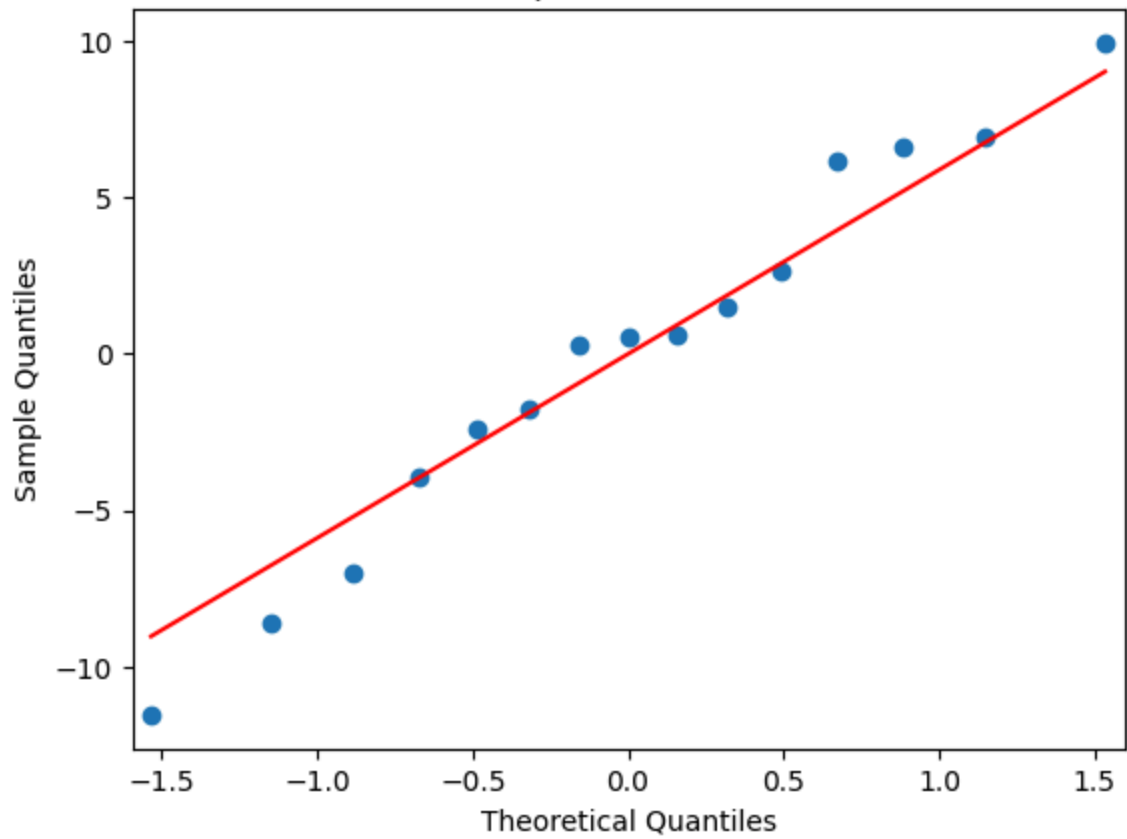
```

```
# Visualización: Q-Q plot
sm.qqplot(residuales, line='s')
plt.title("Q-Q plot de residuos")
plt.show()

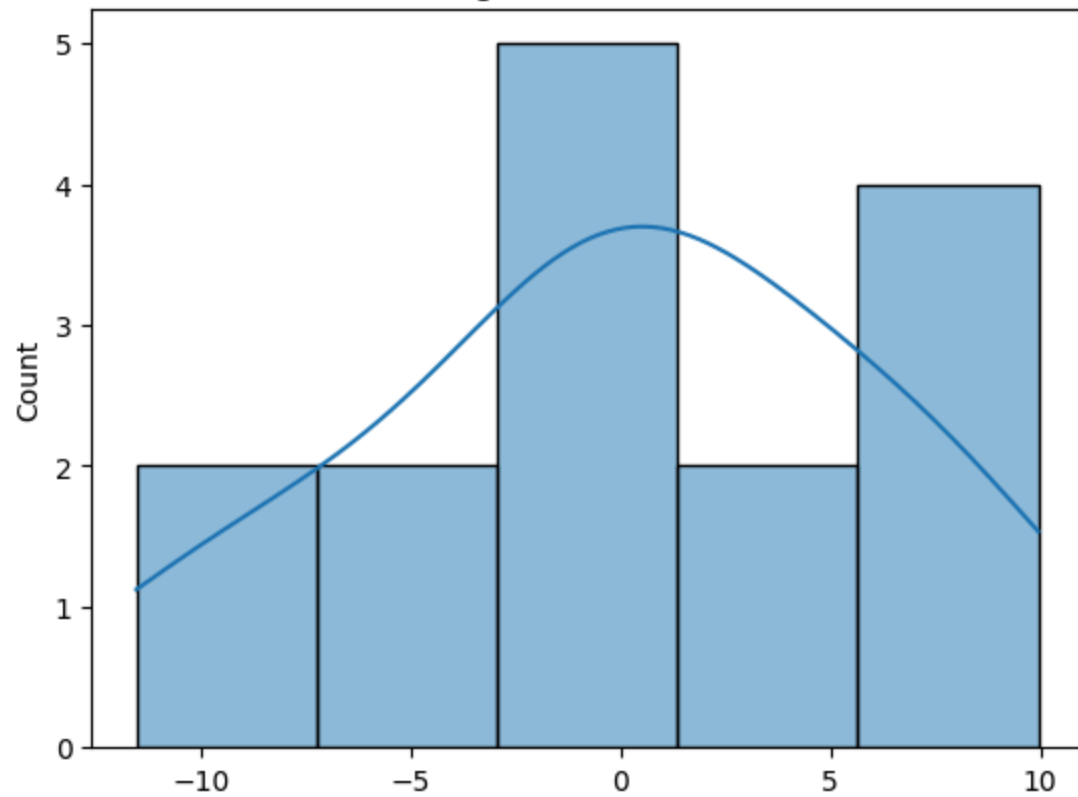
# Histograma
sns.histplot(residuales, kde=True)
plt.title("Histograma de residuos")
plt.show()
```

valor-p (Shapiro) = 0.901827735700704

Q-Q plot de residuos



Histograma de residuos



- ✓ 10. Realice la prueba de Breush-Pagan para los residuales y comente el resultado

```
# Test de Breusch-Pagan
# H0: No hay heterocedasticidad (dispersión es igual en toda la recta)
# H1: Hay Heterocedasticidad (dispersión distinta a lo largo de la recta)
# alpha = 0.05

from statsmodels.stats.api import het_breuschpagan
import statsmodels.api as sm
x_constante = sm.add_constant(X)
_, valor_p_bp, _, _ = het_breuschpagan(residuales, x_constante)
print(f'valor-p de Breusch-Pagan: {valor_p_bp: 0.4f}\n')
```

valor-p de Breusch-Pagan: 0.2289

- ✓ 12. Realice una tabla ANOVA e interprete el resultado.

```
# 12. Realice una tabla ANOVA e interprete el resultado.

# test de ANOVA (Analysis of Variance)
# H0: beta_1 = 0 (No hay correlación)
# H1: beta_1 ≠ 0 (Sí hay correlación)

import statsmodels.api as sm
from statsmodels.formula.api import ols
# Y ~ X
modelo_lineal = ols(
    '''Promedio_final ~ Promedio_exámenes_cortos''',
    data = df).fit()
tabla_anova = sm.stats.anova_lm(modelo_lineal)
tabla_anova
```

	df	sum_sq	mean_sq	F	PR(>F)
Promedio_exámenes_cortos	1.0	1538.228959	1538.228959	38.492412	0.000032
Residual	13.0	519.504375	39.961875	NaN	NaN

Próximos pasos:

[Generar código con tabla_anova](#)

[New interactive sheet](#)

- ✓ Problema 2

William Hawkins, vicepresidente de personal de la International Motors, trabaja en la relación entre el salario de un trabajador y el porcentaje de ausentismo. Hawkins dividió el intervalo de salarios de International en 12 grados o niveles (1 es el menor grado, 12 el más alto) y después muestreó aleatoriamente a un grupo de trabajadores. Determinó el grado de salario de cada trabajador y el número de días que ese empleado había faltado en los últimos 3 años.

Categoría de salario	11	10	8	5	9	7	3
Ausencias	18	17	29	36	11	28	35
Categoría de salario	11	8	7	2	9	8	3
Ausencias	14	20	32	39	16	31	40

```
import pandas as pd
df=pd.DataFrame({
    'Categoría salario':[11,10,8,5,9,7,3,11,8,7,2,9,8,3],
    'Ausencias':[18,17,29,36,11,28,35,14,20,32,39,16,31,40],
})
df
```

	Categoría salario	Ausencias	
0	11	18	
1	10	17	
2	8	29	
3	5	36	
4	9	11	
5	7	28	
6	3	35	
7	11	14	
8	8	20	
9	7	32	
10	2	39	
11	9	16	
12	8	31	
13	3	40	

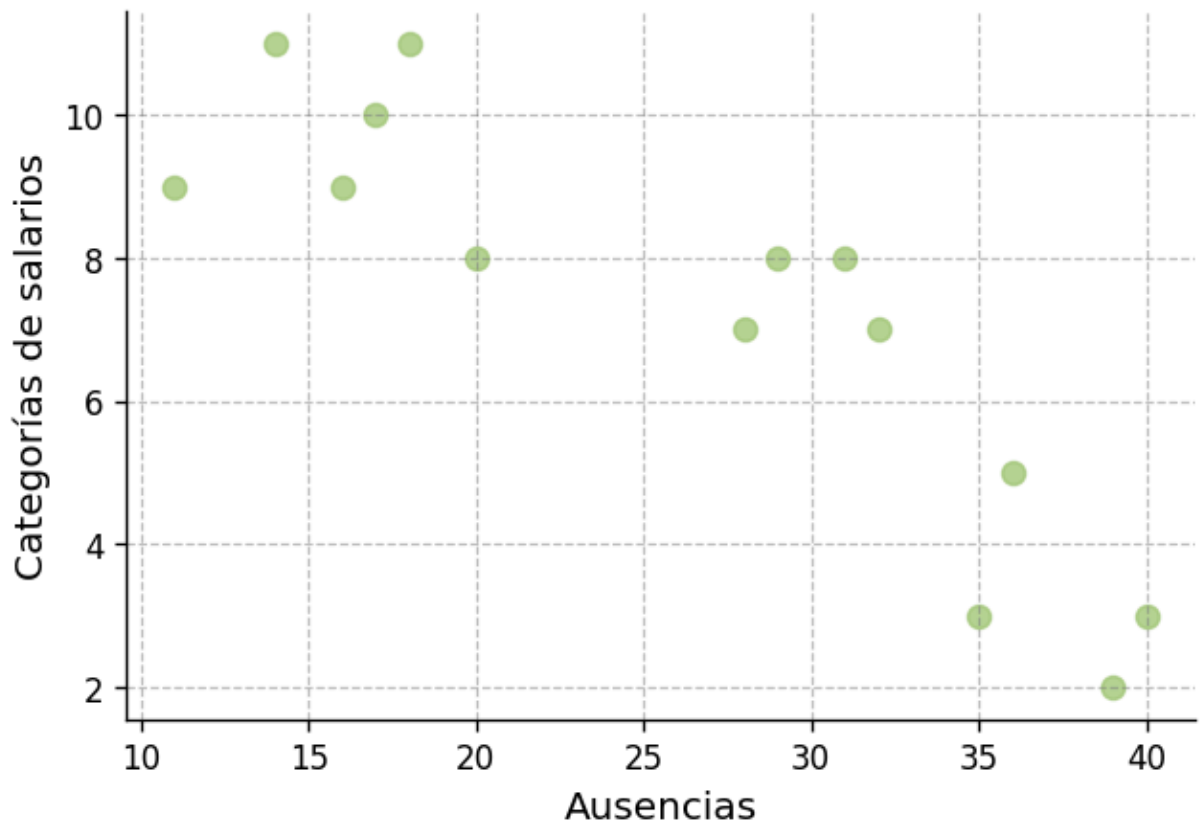
- ✓ 1.Establesca una variable dependiente (Y) y una variable independiente (X).

```
Y =df['Categoría salario']  
X =df['Ausencias']
```

- ✓ 2.Realice un diagrama de dispersión para estos datos

```
# Importamos la librería matplotlib.pyplot y le damos el pseudónimo plt  
import matplotlib.pyplot as plt  
  
# Configuración general del gráfico  
plt.figure(  
    figsize=(6, 4),    # tamaño de la figura (ancho, alto) en pulgadas  
    dpi=120            # resolución del gráfico  
)  
  
# Gráfico de dispersión  
plt.scatter(  
    X, Y,  
    marker="o",        # forma: googlear "matplotlib.markers"  
    color='#A4C979',    # color de los puntos  
    edgecolor='#A4C979', # borde de los puntos  
    alpha=0.8,          # transparencia: 0 es invisible y 1 es color s  
    s=50,               # tamaño de los puntos  
)  
  
# Etiquetas de los ejes  
# eje x  
plt.xlabel(  
    'Ausencias', # etiqueta del eje x  
    fontsize=12  # tamaño de fuente  
)  
  
plt.ylabel(  
    'Categorías de salarios', # etiqueta del eje y  
    fontsize=12 # tamaño de fuente  
)  
  
# Fuente de los ticks  
plt.xticks(fontsize=10)  
plt.yticks(fontsize=10)  
  
# Márgenes  
plt.gca().spines['right'].set_visible(False) # derecha  
plt.gca().spines['top'].set_visible(False)   # superior
```

```
# Cuadrícula (opcional, pero didáctica)
plt.grid(
    visible=True,
    linestyle='--',
    linewidth=0.7,
    alpha=0.5,
    color="gray"
)
```



✓ 3.¿Los datos soportan la suposición de linealidad?

El gráfico de dispersión muestra una clara tendencia lineal. Se puede observar que conforme aumentan las horas de estudio (X) aumenta el índice de desemeño (Y). Con lo que se espera un coeficiente de correlación positivo y parece cumplirse el supuesto de linealidad.

✓ 4.Calcule el coeficiente de correlación e interprete los resultados

```
# Importar la función pearsonr desde scipy.stats
from scipy.stats import pearsonr
```

```
# Test de Pearson
# H0: rho = 0      (No hay correlación)
# H1: rho ≠ 0     (Sí hay correlación)
# alpha = 0.05

r, valor_p = pearsonr(X,Y)

print(f"Coeficiente de correlación: {r: 0.4f}")
print(f'valor_p: {valor_p: 0.4f}')
```

```
Coeficiente de correlación: -0.8801
valor_p: 0.0000
```

El coeficiente de correlación es $r = -0.8801$, este valor indica que la correlación es fuerte, dado que está muy cerca de 1. Además, el que sea negativa, implica que aumentar las ausencias (X) disminuye el salario (Y).

✓ 5. Calcule el coeficiente de determinación e interprete los resultados

```
import statsmodels.api as sm
x_constante = sm.add_constant(X)
modelo = sm.OLS(Y, x_constante).fit()
y_calculada = modelo.predict(x_constante)
```

```
# Coeficiente de determinación
from sklearn.metrics import r2_score

r2 = r2_score(Y, y_calculada)

print(f"Coeficiente de determinación: {r2: 0.2%}")
```

```
Coeficiente de determinación: 77.46%
```

Con coeficiente de determinación de

$$r^2 = 0.7746$$

indica que con las ausencias estoy justificando en un 77.46% la variabilidad en los salarios. Por lo que el modelo puede tener una buena capacidad de predicción.

✓ 6. Obtenga la recta de regresión ajustada y gráfiquelo sobre el gráfico de dispersión

```
import statsmodels.api as sm
x_constante = sm.add_constant(X)
modelo=sm.OLS(Y,x_constante).fit()
y_calculada=modelo.predict(x_constante)

modelo.params
```

	0
const	14.058843
Ausencias	-0.261814

dtype: float64

El modelo es:

$$\hat{y} = 14.0588 - 0.2618x$$

donde x es el factor de ausencias

7. Obtenga un intervalo de confianza del 95% para la pendiente de la recta de regresión ajustada (b_1)

```
#intervalos de confianza
modelo.conf_int(alpha=0.05)
```

	0	1
const	11.587445	16.530241
Ausencias	-0.350638	-0.172989

```
# Importamos la librería matplotlib.pyplot y le damos el pseudónimo plt
import matplotlib.pyplot as plt
```

```
# Configuración general del gráfico
plt.figure(
    figsize=(6, 4),    # tamaño de la figura (ancho, alto) en pulgadas
    dpi=120             # resolución del gráfico
)
```

```
# Gráfico de dispersión
plt.scatter(
    X, Y,
    marker="*",          # forma: googlear "matplotlib.markers"
    color='blue',        # color de los puntos
    edgecolor='#E38DBF', # borde de los puntos
)
```

```

        alpha=0.9,          # transparencia: 0 es invisible y 1 es color sólido
        s=50,              # tamaño de los puntos
    )

plt.plot(
    X, y_calculada,
    color="blue",          # color de la línea de regresión
    alpha=0.9,            # transparencia: 0 es invisible y 1 es color sólido
    label="Recta de regresión" # Etiqueta para la leyenda
)

# Las líneas siguientes se usan típicamente en gráficos de residuales, no
# Si quieres ver la línea de regresión con la etiqueta, usa plt.legend()
# plt.axhline(
#     y=0,          # Donde está la línea horizontal
#     label="Recta de regresión", # Etiqueta
#     linestyle = "--", # Estilo de línea
#     color = "black", # color
#     linewidth=1.0   # ancho de línea
# )

# Etiquetas de los ejes
plt.xlabel(
    'Promedio exámenes cortos', # etiqueta del eje x
    fontsize=12 # tamaño de fuente
)

plt.ylabel(
    'Promedio final', # etiqueta del eje y (cambiado de 'Residuales' y '
    fontsize=12 # tamaño de fuente
)

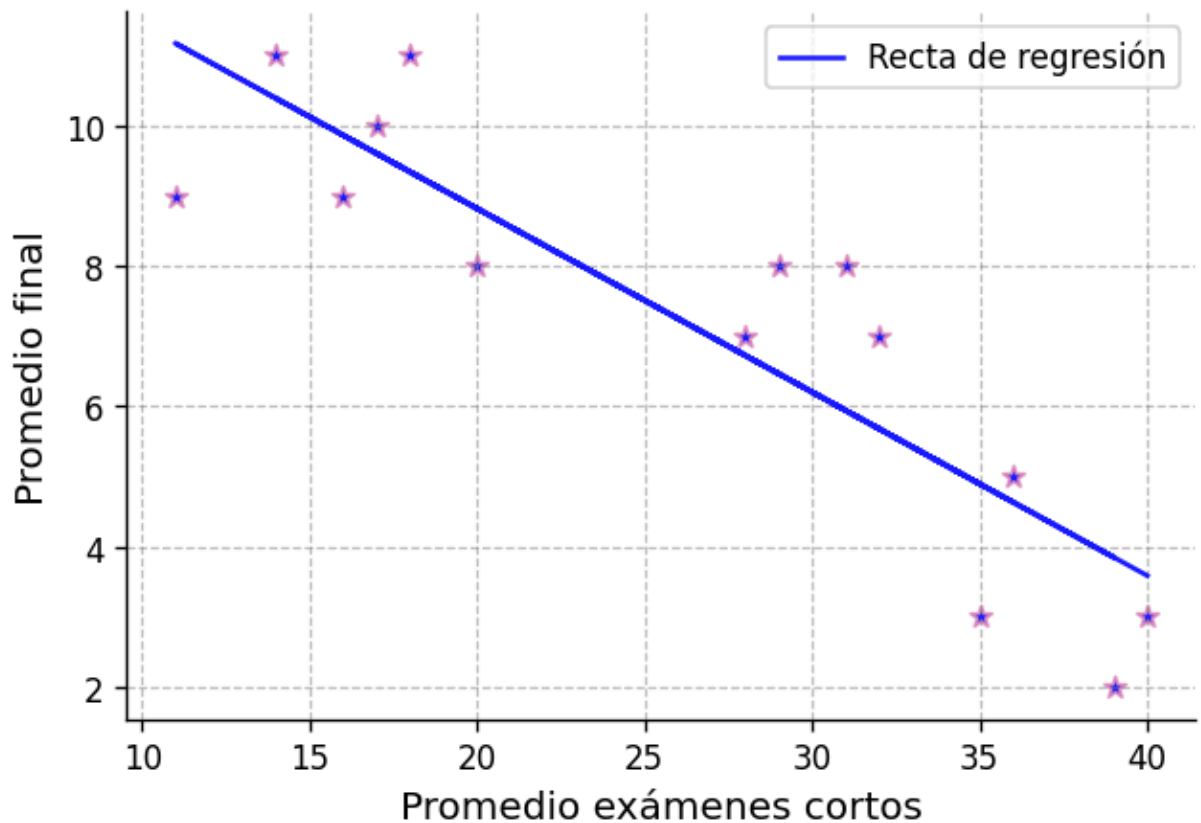
# Fuente de los ticks
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)

# Márgenes
plt.gca().spines['right'].set_visible(False) # derecha
plt.gca().spines['top'].set_visible(False)   # superior

# Cuadrícula (opcional, pero didáctica)
plt.grid(
    visible=True,
    linestyle='--',
    linewidth=0.7,
    alpha=0.5,
    color="gray"
)

plt.legend() # Muestra la leyenda con la etiqueta de la recta de regresión
plt.show() # Muestra el gráfico

```



El verdadero b_0 está entre 11.587445 y 16.530241 con una confianza del 95%. Esto indica que una persona sin ausencias ($X = 0$) tiene una categoría de salario (Y) que está entre estos dos valores.

El verdadero valor de b_1 está entre -0.350638 y -0.172989 con un 95% de confianza. Dado que dentro de este intervalo no está el valor de cero, entonces la relación es significativa. También indica que por cada falta se pierde entre 0.172989 y 0.350638 en la categoría de salario.

8. Calcule los residuales y trace un nuevo gráfico de dispersión.

Comente, ¿Parece que se verifican los supuestos?

```
residuales = modelo.resid
```

```
# Importamos la librería matplotlib.pyplot y le damos el pseudónimo plt
import matplotlib.pyplot as plt
```

```
# Configuración general del gráfico
```

```
plt.figure(
    figsize=(6, 4), # tamaño de la figura (ancho, alto) en pulgadas
```

```

    dpi=120                # resolución del gráfico
)

# Gráfico de dispersión de residuales
plt.scatter(
    X, residuales,
    marker="o",            # forma: googlear "matplotlib.markers"
    color='#A4C979',       # color de los puntos
    edgecolor='#A4C979',   # borde de los puntos
    alpha=0.9,             # transparencia: 0 es invisible y 1 es color s
    s=50,                  # tamaño de los puntos
)

# Línea horizontal en y=0 para el gráfico de residuales
plt.axhline(
    y=0,
    color='black',
    linewidth=1.0,
    linestyle='--',
    label='y=0'
)

# Etiquetas de los ejes
# eje x
plt.xlabel(
    'Promedio exámenes cortos', # etiqueta del eje x
    fontsize=12 # tamaño de fuente
)

plt.ylabel(
    'Residuales', # etiqueta del eje y
    fontsize=12 # tamaño de fuente
)

# Leyenda
plt.legend(
    fontsize=10,
    loc='best',
    frameon=True
)

# Fuente de los ticks
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)

# Márgenes
plt.gca().spines['right'].set_visible(False) # derecha
plt.gca().spines['top'].set_visible(False)   # superior

# Cuadrícula (opcional, pero didáctica)
plt.grid(

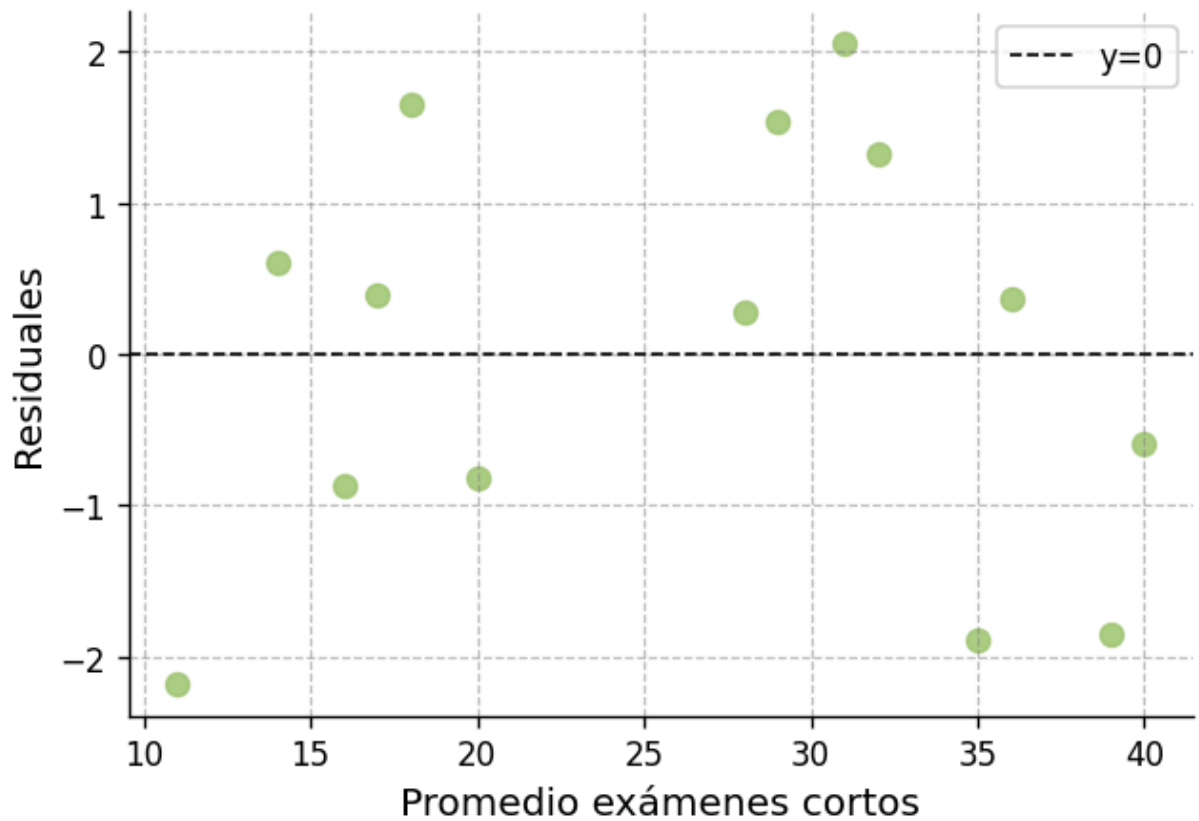
```

```

visible=True,
linestyle='--',
linewidth=0.7,
alpha=0.5,
color="gray"
)

plt.show() # Muestra el gráfico

```



Linealidad :

Se cumple, ya que los residuos se distribuyen aleatoriamente alrededor del eje cero, presentando valores tanto positivos como negativos. Asimismo, no se identifican curvaturas ni patrones sistemáticos que indiquen que un modelo no lineal describiría mejor la relación entre las variables.

Normalidad :

Se cumple, ya que, aunque el tamaño de la muestra es reducido, los errores presentan una distribución aproximadamente simétrica alrededor del cero, con una proporción equilibrada de residuos positivos y negativos, lo que es consistente con el supuesto de normalidad.

Homoscedasticidad :

Se cumple debido a que la dispersión vertical de los residuos permanece relativamente constante a lo largo de todo el rango de los valores ajustados. La ausencia de cambios significativos en la amplitud de dicha dispersión indica que la varianza de los errores es constante.

Independencia :

Se cumple, ya que los residuos no presentan tendencias, secuencias, rachas continuas ni patrones cíclicos que sugieran dependencia entre observaciones consecutivas. En conjunto, los errores muestran un comportamiento aleatorio, respaldando el supuesto de independencia.

9. Realice la prueba de Shapiro para los residuales y comente el resultado.

```
from scipy.stats import shapiro
import matplotlib.pyplot as plt
import seaborn as sns

#test de shapiro
#H0: Hay normalidad
#H1: No hay normalidad

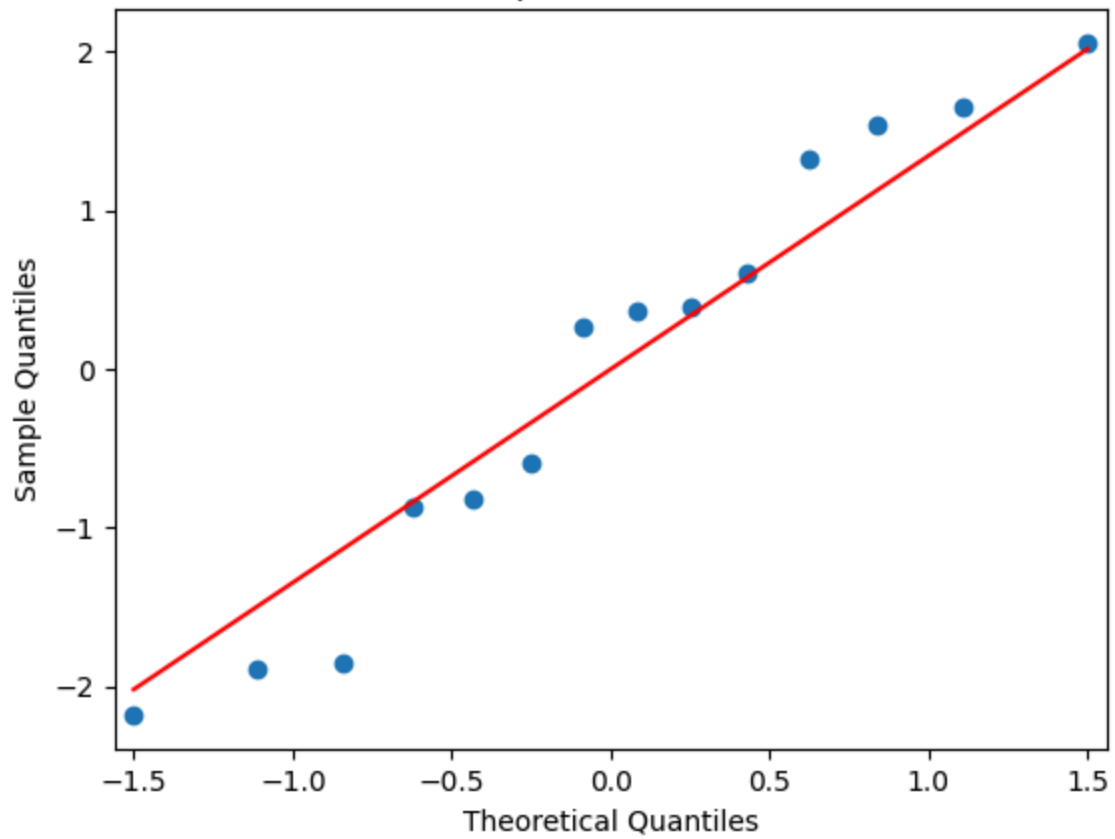
# Prueba de Shapiro-Wilk
stat, valor_p_sh = shapiro(residuales)
print(f"valor-p (Shapiro) = {valor_p_sh}")

# Visualización: Q-Q plot
sm.qqplot(residuales, line='s')
plt.title("Q-Q plot de residuos")
plt.show()

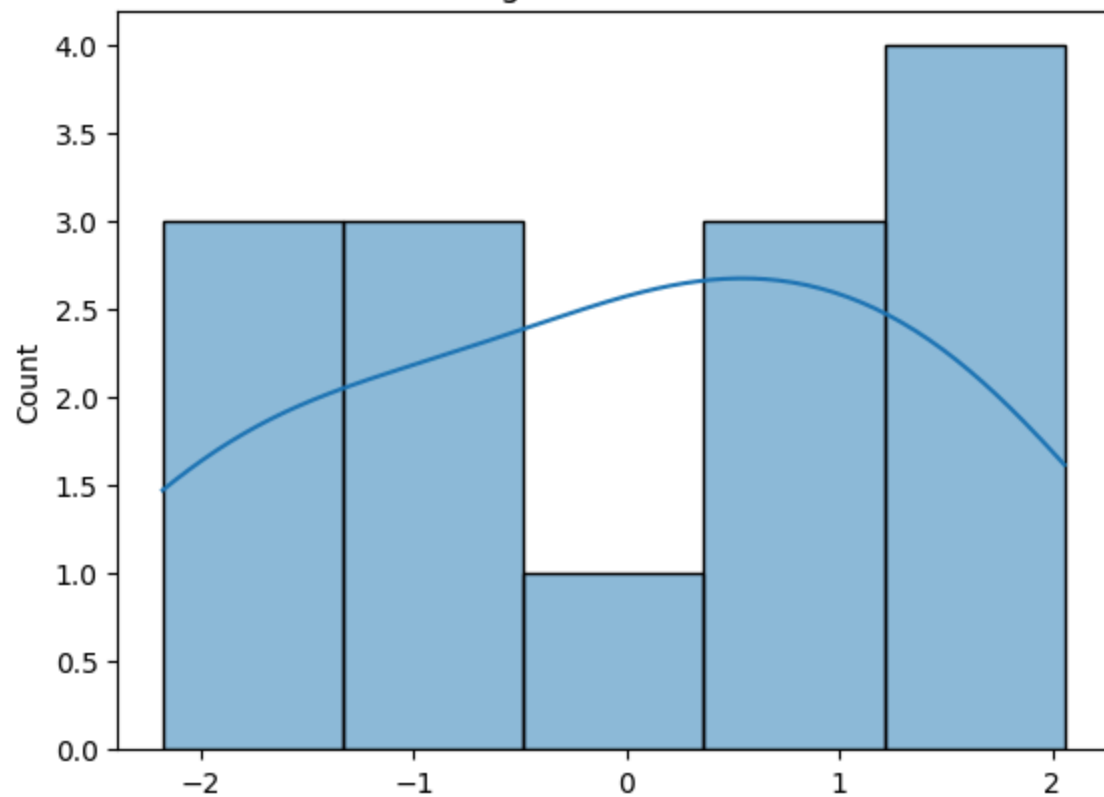
# Histograma
sns.histplot(residuales, kde=True)
plt.title("Histograma de residuos")
plt.show()
```

valor-p (Shapiro) = 0.4172971767713699

Q-Q plot de residuos



Histograma de residuos



- ✓ 10. Realice la prueba de Breusch-Pagan para los residuales y comente el resultado.

```
# Test de Breusch-Pagan
# H0: No hay heterocedasticidad (dispersión es igual en toda la recta)
# H1: Hay Heterocedasticidad (dispersión distinta a lo largo de la recta)
# alpha = 0.05

from statsmodels.stats.api import het_breuschpagan
import statsmodels.api as sm
x_constante = sm.add_constant(X)
_, valor_p_bp, _, _ = het_breuschpagan(residuales, x_constante)
print(f'valor-p de Breusch-Pagan: {valor_p_bp: 0.4f}\n')
```

```
valor-p de Breusch-Pagan: 0.8955
```

En la prueba de Breusch-Pagan se obtuvo un valor $p = 0.8955$. Dado que este valor es mayor que el nivel de significancia de $\alpha = 0.05$, no se rechaza la hipótesis nula (H_0) de homocedasticidad. Por lo tanto, no existe evidencia estadísticamente significativa de que la varianza de los errores sea no constante. En consecuencia, se concluye que se cumple el supuesto de homocedasticidad, indicando que la dispersión de los residuales permanece aproximadamente constante a lo largo de los valores ajustados del modelo.

- ✓ 11. Utiliza la recta de regresión para interpolar dos valores y extrapolar uno. Comenta estos resultados.

```
# minimo ausencias: 11
# maximo ausencias: 40

# interpolar , valores de x entre 11 y 40

# interpolar 1
y_25 = modelo.predict([1, 25])
y_32 = modelo.predict([1, 32])

print(f"Para x = 25, y = {y_25[0]: 0.4f}")
print(f"Para x = 32, y = {y_32[0]: 0.4f}")
```

```
Para x = 25, y = 7.5135
Para x = 32, y = 5.6808
```

```
df["salario_predicho"] = y_calculada.round()
df
```

	Categoría	salario	Ausencias	salario_predicho	
0		11	18	9.0	
1		10	17	10.0	
2		8	29	6.0	
3		5	36	5.0	
4		9	11	11.0	
5		7	28	7.0	
6		3	35	5.0	
7		11	14	10.0	
8		8	20	9.0	
9		7	32	6.0	
10		2	39	4.0	
11		9	16	10.0	
12		8	31	6.0	
13		3	40	4.0	

Próximos pasos:

[Generar código con df](#)

[New interactive sheet](#)

```
# mínimo ausencias: 11
# máximo ausencias: 40

# Extrapolar, valores de x que no estén entre 11 y 40

y_10 = modelo.predict([[1, 10]])

print(f"Para x = 10, y = {y_10[0]: 0.4f}")
```

Para x = 10, y = 11.4407

✓ 12. Realice una tabla ANOVA e interprete el resultado.

```
# 12. Realice una tabla ANOVA e interprete el resultado.

# test de ANOVA (Analysis of Variance)
# H0: beta_1 = 0 (No hay correlación)
# H1: beta_1 ≠ 0 (Sí hay correlación)
```

```
import statsmodels.api as sm
from statsmodels.formula.api import ols
# Y ~ X
modelo_lineal = ols(
    '''Q('Categoría salario') ~ Ausencias''',
    data = df).fit()
tabla_anova = sm.stats.anova_lm(modelo_lineal)
tabla_anova
```

	df	sum_sq	mean_sq	F	PR(>F)	
Ausencias	1.0	87.034348	87.034348	41.243954	0.000033	
Residual	12.0	25.322795	2.110233	NaN	NaN	

Próximos pasos:

[Generar código con tabla_anova](#)

[New interactive sheet](#)

En la tabla ANOVA se obtuvo un estadístico $F = 41.243954$ con un valor $p=0.000033$. Como el valor p es menor que el nivel de significancia de 0.05, se rechaza la hipótesis nula ($H_0 : \beta_1 = 0$). Por lo tanto, existe evidencia estadísticamente significativa para concluir que la pendiente de la recta de regresión es diferente de cero y que existe una relación lineal entre las ausencias y la categoría salarial. En consecuencia, el modelo de regresión lineal es estadísticamente significativo y resulta adecuado para explicar la variabilidad de la variable respuesta.

✓ Problema 3

A menudo, quienes hacen la contabilidad de costos estiman los gastos generales con base en el nivel de producción. En Standard Knitting Co. han reunido información acerca de los gastos generales y las unidades producidas en diferentes plantas.

Gastos generales	191	170	272	155	280	173	234	116	153	178
Unidades	40	42	53	35	56	39	48	30	37	40

```
import pandas as pd
df=pd.DataFrame({
    'Gastos generales':[191,170,272,155,280,173,234,116,153,178],
    'Unidades':[40,42,53,35,56,39,48,30,37,40]
})
df
```

	Gastos generales	Unidades	
0	191	40	
1	170	42	
2	272	53	
3	155	35	
4	280	56	
5	173	39	
6	234	48	
7	116	30	
8	153	37	
9	178	40	

Próximos pasos:

[Generar código con df](#)

[New interactive sheet](#)

1.Establesca una variable dependiente (Y) y una variable independiente (X).

```
Y =df['Gastos generales']
X =df['Unidades']
```

2.Realice un diagrama de dispersión para estos datos

```
# Importamos la librería matplotlib.pyplot y le damos el pseudónimo plt
import matplotlib.pyplot as plt

# Configuración general del gráfico
plt.figure(
    figsize=(6, 4),    # tamaño de la figura (ancho, alto) en pulgadas
    dpi=120            # resolución del gráfico
)

# Gráfico de dispersión
plt.scatter(
    X, Y,
    marker="o",        # forma: googlear "matplotlib.markers"
    color='#84C6DB',   # color de los puntos
    edgecolor='#84C6DB', # borde de los puntos
    alpha=0.8,         # transparencia: 0 es invisible y 1 es color s
```

```

        s=50,                # tamaño de los puntos
    )

# Etiquetas de los ejes
# eje x
plt.xlabel(
    'Unidades', # etiqueta del eje x
    fontsize=12 # tamaño de fuente
)

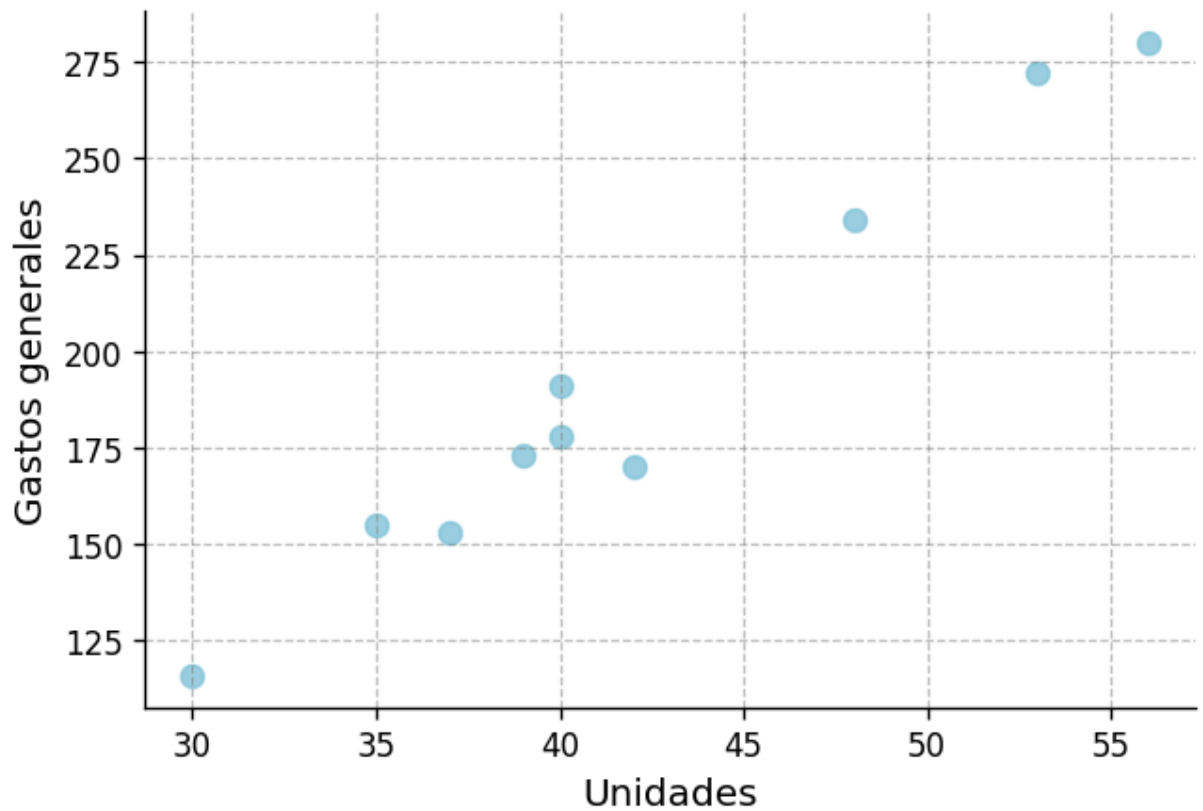
plt.ylabel(
    'Gastos generales', # etiqueta del eje y
    fontsize=12 # tamaño de fuente
)

# Fuente de los ticks
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)

# Márgenes
plt.gca().spines['right'].set_visible(False) # derecha
plt.gca().spines['top'].set_visible(False)   # superior

# Cuadrícula (opcional, pero didáctica)
plt.grid(
    visible=True,
    linestyle='--',
    linewidth=0.7,
    alpha=0.5,
    color="gray"
)

```



✓ 3.¿Los datos soportan la suposición de linealidad?

Sí, los datos soportan la suposición de linealidad. El gráfico de dispersión muestra una clara tendencia lineal. Se puede observar que conforme aumentan las unidades (X), aumentan los gastos generales (Y). Con ello, se espera un coeficiente de correlación positivo y parece cumplirse el supuesto de linealidad.

✓ 4.Calcule el coeficiente de correlación e interprete los resultados

```
# Importar la función pearsonr desde scipy.stats
from scipy.stats import pearsonr

# Test de Pearson
# H0: rho = 0      (No hay correlación)
# H1: rho ≠ 0     (Sí hay correlación)
# alpha = 0.05

r, valor_p = pearsonr(X,Y)

print(f"Coeficiente de correlación: {r: 0.4f}")
print(f'valor_p: {valor_p: 0.4f}')
```

```
Coeficiente de correlación:  0.9835
valor_p:  0.0000
```

El coeficiente de correlación es ($r = 0.9835$), lo que indica que existe una **correlación lineal positiva muy fuerte** entre las variables, ya que su valor es muy cercano a 1. Además, al ser positivo, significa que, a medida que aumenta la variable (X), también aumenta la variable (Y). Por otra parte, el **valor p = 0.0000** es menor que el nivel de significancia ($\alpha = 0.05$), por lo que **se rechaza la hipótesis nula (H_0)** y se concluye que **existe una correlación lineal estadísticamente significativa** entre ambas variables.

✓ 5. Calcule el coeficiente de determinación e interprete los resultados

```
import statsmodels.api as sm
x_constante = sm.add_constant(X)
modelo = sm.OLS(Y, x_constante).fit()
y_calculada = modelo.predict(x_constante)
```

```
# Coeficiente de determinación
from sklearn.metrics import r2_score

r2 = r2_score(Y, y_calculada)

print(f"Coeficiente de determinación: {r2: 0.2%}")
```

```
Coeficiente de determinación:  96.73%
```

Con un coeficiente de determinación de:

$$R^2 = 0.9673$$

se indica que las Unidades explican el 96.73 de la variabilidad en los Gastos generales. Por lo tanto, el modelo presenta una buena capacidad de predicción, ya que la mayor parte de la variación en los gastos generales puede explicarse a partir de las unidades. El 3.27% restante de la variabilidad se atribuye a otros factores no incluidos en el modelo o al error aleatorio.

6. Obtenga la recta de regresión ajustada y gráfiquelo sobre el gráfico de dispersión

```
import statsmodels.api as sm
x_constante = sm.add_constant(X)
modelo=sm.OLS(Y,x_constante).fit()
y_calculada=modelo.predict(x_constante)

modelo.params
```

	0
const	-80.442857
Unidades	6.491497
dtype:	float64

El modelo es:

$$\hat{y} = -80.442857 + 6.491497x$$

donde x son las unidades

7. Obtenga un intervalo de confianza del 95% para la pendiente de la recta de regresión ajustada (b_1)

```
#intervalos de confianza
modelo.conf_int(alpha=0.05)
```

	0	1	
const	-121.986373	-38.899341	
Unidades	5.518450	7.464543	

```

# Importamos la librería matplotlib.pyplot y le damos el pseudónimo plt
import matplotlib.pyplot as plt

# Configuración general del gráfico
plt.figure(
    figsize=(6, 4),    # tamaño de la figura (ancho, alto) en pulgadas
    dpi=120            # resolución del gráfico
)

# Gráfico de dispersión
plt.scatter(
    X, Y,
    marker="*",        # forma: googlear "matplotlib.markers"
    color='#84C6DB',   # color de los puntos
    edgecolor='#84C6DB', # borde de los puntos
    alpha=0.9,         # transparencia: 0 es invisible y 1 es color s
    s=50,              # tamaño de los puntos
)

plt.plot(
    X, y_calculada,
    color="#81C26E",   # color de la línea de regresión
    alpha=0.9,         # transparencia: 0 es invisible y 1 es color sólí
    label="Recta de regresión" # Etiqueta para la leyenda
)

# Las líneas siguientes se usan típicamente en gráficos de residuales, n
# Si quieres ver la línea de regresión con la etiqueta, usa plt.legend()
# plt.axhline(
#     y=0,            # Donde está la línea horizontal
#     label="Recta de regresión", # Etiqueta
#     linestyle = "--", # Estilo de línea
#     color = "black", # color
#     linewidth=1.0   # ancho de línea
# )

# Etiquetas de los ejes
plt.xlabel(
    'Promedio exámenes cortos', # etiqueta del eje x
    fontsize=12 # tamaño de fuente
)

plt.ylabel(
    'Promedio final', # etiqueta del eje y (cambiado de 'Residuales' y '
    fontsize=12 # tamaño de fuente
)

# Fuente de los ticks
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)

```

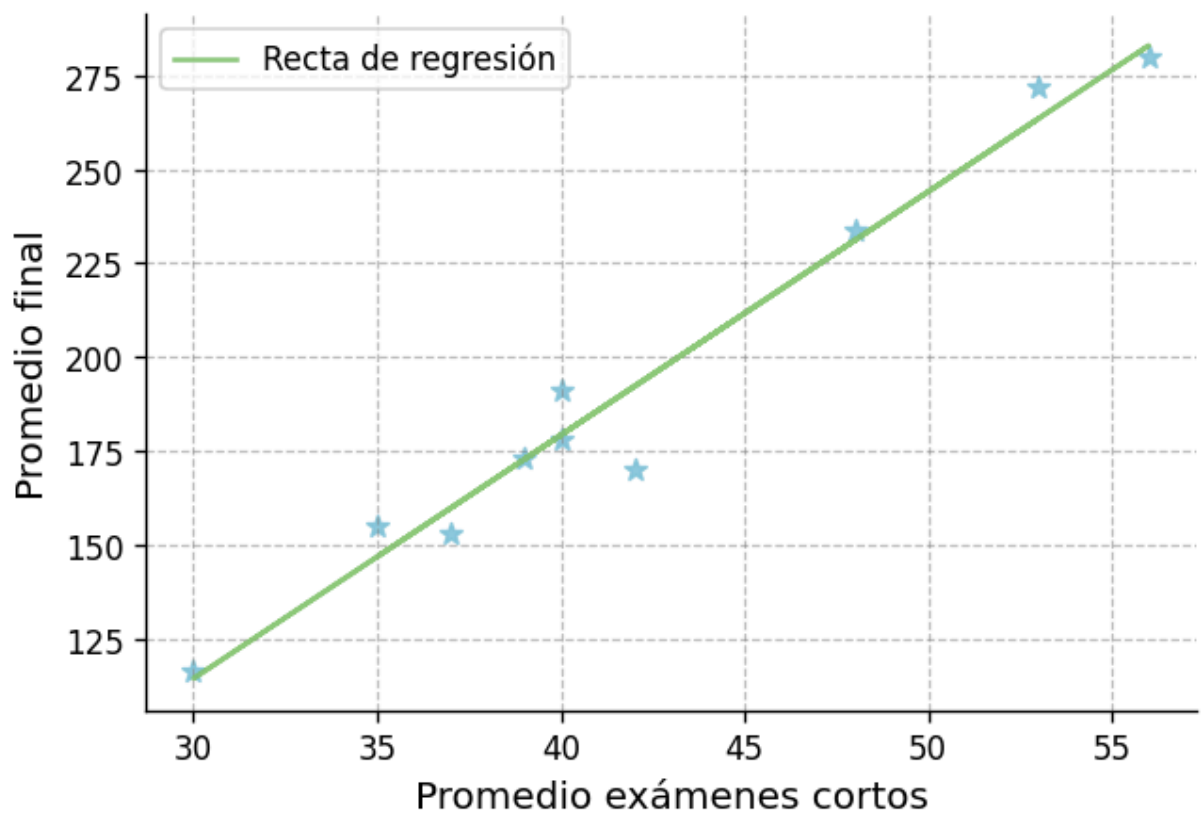
```

# Márgenes
plt.gca().spines['right'].set_visible(False) # derecha
plt.gca().spines['top'].set_visible(False)   # superior

# Cuadrícula (opcional, pero didáctica)
plt.grid(
    visible=True,
    linestyle='--',
    linewidth=0.7,
    alpha=0.5,
    color="gray"
)

plt.legend() # Muestra la leyenda con la etiqueta de la recta de regresión
plt.show() # Muestra el gráfico

```



8. Calcule los residuales y trace un nuevo gráfico de dispersión.

Comente, ¿Parece que se verifican los supuestos?

```
residuales = modelo.resid
```

```

# Importamos la librería matplotlib.pyplot y le damos el pseudónimo plt
import matplotlib.pyplot as plt

```

```

# Configuración general del gráfico
plt.figure(
    figsize=(6, 4),    # tamaño de la figura (ancho, alto) en pulgadas
    dpi=120             # resolución del gráfico
)

# Gráfico de dispersión de residuales
plt.scatter(
    X, residuales,
    marker="o",          # forma: googlear "matplotlib.markers"
    color='#C28DE3',     # color de los puntos
    edgecolor='#DDA6FF', # borde de los puntos
    alpha=0.9,           # transparencia: 0 es invisible y 1 es color s
    s=50,                # tamaño de los puntos
)

# Línea horizontal en y=0 para el gráfico de residuales
plt.axhline(
    y=0,
    color='black',
    linewidth=1.0,
    linestyle='--',
    label='y=0'
)

# Etiquetas de los ejes
# eje x
plt.xlabel(
    'Promedio exámenes cortos', # etiqueta del eje x
    fontsize=12 # tamaño de fuente
)

plt.ylabel(
    'Residuales', # etiqueta del eje y
    fontsize=12 # tamaño de fuente
)

# Leyenda
plt.legend(
    fontsize=10,
    loc='best',
    frameon=True
)

# Fuente de los ticks
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)

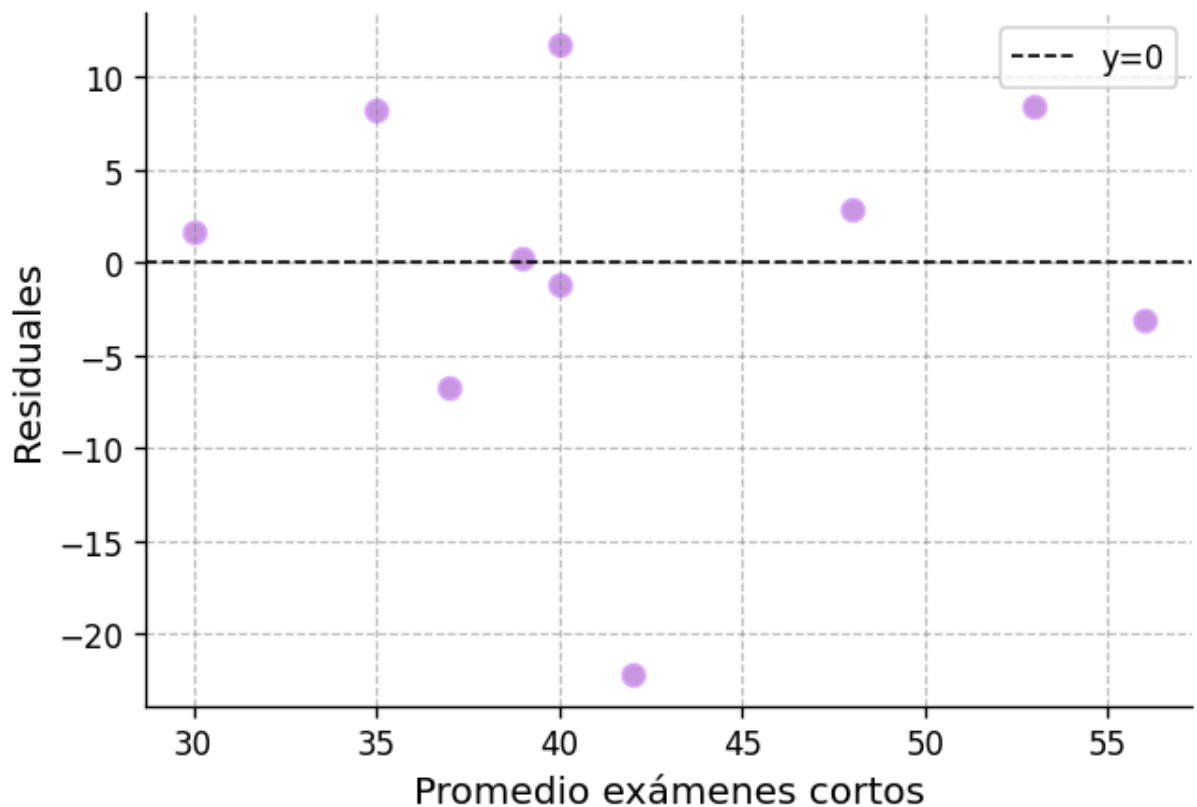
# Márgenes
plt.gca().spines['right'].set_visible(False) # derecha

```

```
plt.gca().spines['top'].set_visible(False) # superior

# Cuadrícula (opcional, pero didáctica)
plt.grid(
    visible=True,
    linestyle='--',
    linewidth=0.7,
    alpha=0.5,
    color="gray"
)

plt.show() # Muestra el gráfico
```



Linealidad :

Se cumple, ya que los residuos se distribuyen aleatoriamente alrededor del eje cero, presentando valores tanto positivos como negativos. Además, no se observan curvaturas ni patrones que indiquen un modelo no lineal.

Normalidad :

Se cumple de manera razonable, ya que los residuos muestran una distribución aproximadamente alrededor del cero. Aunque se observa un residual con mayor magnitud, en general no existen evidencias de desviaciones importantes respecto al supuesto de normalidad.

Homoscedasticidad :

Se cumple, debido a que la dispersión vertical de los residuos permanece constante a lo largo de los distintos valores del eje X de gastos generales. No se aprecia un patrón de ensanchamiento o estrechamiento que indique una varianza no constante.

Independencia :

Se cumple, ya que los residuos no presentan tendencias, secuencias, rachas continuas ni patrones que sugieran dependencia entre las generaciones. En conjunto, los errores muestran un comportamiento aleatorio, respaldando el supuesto de independencia.

9. Realice la prueba de Shapiro para los residuales y comente el resultado.

```
from scipy.stats import shapiro
import matplotlib.pyplot as plt
import seaborn as sns

#test de shapiro
#H0: Hay normalidad
#H1: No hay normalidad

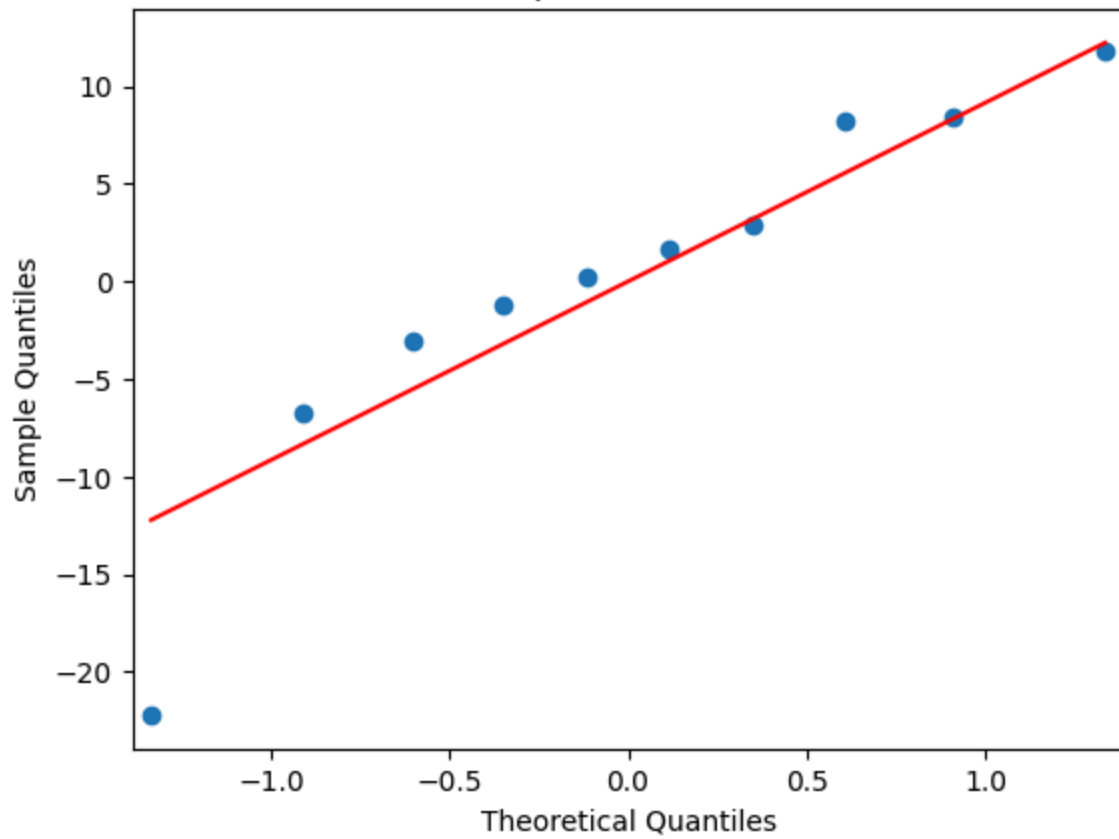
# Prueba de Shapiro-Wilk
stat, valor_p_sh = shapiro(residuales)
print(f"valor-p (Shapiro) = {valor_p_sh}")

# Visualización: Q-Q plot
sm.qqplot(residuales, line='s')
plt.title("Q-Q plot de residuos")
plt.show()

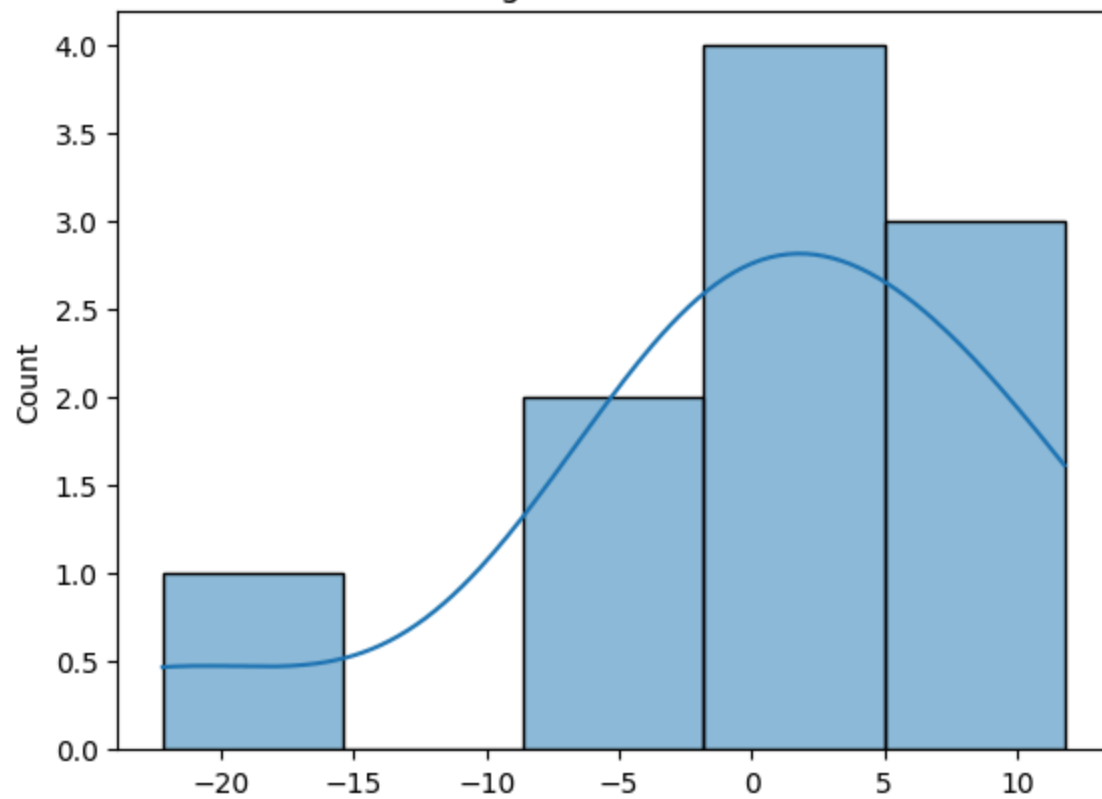
# Histograma
sns.histplot(residuales, kde=True)
plt.title("Histograma de residuos")
plt.show()
```

valor-p (Shapiro) = 0.22045077017698533

Q-Q plot de residuos



Histograma de residuos



- ✓ 10. Realice la prueba de Breusch-Pagan para los residuales y comente el resultado.

```
# Test de Breusch-Pagan
# H0: No hay heterocedasticidad (dispersión es igual en toda la recta)
# H1: Hay Heterocedasticidad (dispersión distinta a lo largo de la recta)
# alpha = 0.05

from statsmodels.stats.api import het_breuschpagan
import statsmodels.api as sm
x_constante = sm.add_constant(X)
_, valor_p_bp, _, _ = het_breuschpagan(residuales, x_constante)
print(f'valor-p de Breusch-Pagan: {valor_p_bp: 0.4f}\n')
```

valor-p de Breusch-Pagan: 0.9858

Se obtuvo un valor $p = 0.9858$. Dado que este valor es mayor que el nivel de significancia de $\alpha = 0.05$, no se rechaza la hipótesis nula (H_0) de homocedasticidad. Por lo tanto, no existe evidencia estadísticamente significativa de heterocedasticidad en los residuales. En consecuencia, se concluye que se cumple el supuesto de homocedasticidad, indicando que la varianza de los errores permanece aproximadamente constante a lo largo de los valores ajustados del modelo.

- ✓ 11. Utiliza la recta de regresión para interpolar dos valores y extrapolar uno. Comenta estos resultados.

```
# minimo unidades: 30
# maximo unidades: 56

# interpolar , valores de x entre 30 y 56

# interpolar 1
y_35 = modelo.predict([1, 35])
y_40 = modelo.predict([1, 40])

print(f"Para x = 35, y = {y_35[0]: 0.4f}")
print(f"Para x = 40, y = {y_40[0]: 0.4f}")
```

Para x = 35, y = 146.7595
Para x = 40, y = 179.2170

```
df["Predicción de gastos"] = y_calculada.round()
df
```

	Gastos generales	Unidades	Predicción de gastos	
0	191	40	179.0	
1	170	42	192.0	
2	272	53	264.0	
3	155	35	147.0	
4	280	56	283.0	
5	173	39	173.0	
6	234	48	231.0	
7	116	30	114.0	
8	153	37	160.0	
9	178	40	179.0	

Próximos pasos:

[Generar código con df](#)

[New interactive sheet](#)

```
# mínimo unidades: 30
# máximo unidades: 56

# Extrapolar, valores de x que no estén entre 30 y 56

y_23 = modelo.predict([[1, 23]])

print(f"Para x = 23, y = {y_23[0]: 0.4f}")
```

Para x = 23, y = 68.8616

Interpolación :

Para $x = 35$ unidades producidas, la recta de regresión estima unos gastos generales de $y = 146.7595$. Asimismo, para $x=40$ unidades producidas, el valor estimado es $y = 179.2170$. Como ambos valores de X se encuentran dentro del rango de los datos observados (30 a 56 unidades), estas predicciones corresponden a una interpolación y pueden considerarse razonablemente confiables y consistentes con el comportamiento descrito por el modelo.

Extrapolación :

Para $x = 23$ unidades producidas, la recta de regresión estima unos gastos generales de $y = 68.8616$. Sin embargo, este valor de X se encuentra fuera del rango de los datos

utilizados para ajustar el modelo (30 a 56 unidades), por lo que la predicción debe interpretarse con precaución. Fuera del intervalo observado, la relación entre las variables puede variar y el modelo lineal podría generar estimaciones menos confiables.

✓ 12. Realice una tabla ANOVA e interprete el resultado.

```
import statsmodels.api as sm
from statsmodels.formula.api import ols
# Y ~ X
modelo_lineal = ols(
    '''Q('Gastos generales') ~ Unidades''',
    data = df).fit()
tabla_anova = sm.stats.anova_lm(modelo_lineal)
tabla_anova
```

	df	sum_sq	mean_sq	F	PR(>F)	
Unidades	1.0	24778.042517	24778.042517	236.669535	3.167080e-07	
Residual	8.0	837.557483	104.694685	NaN	NaN	

Próximos pasos:

[Generar código con tabla_anova](#)

[New interactive sheet](#)

En la tabla ANOVA se obtiene un estadístico $F = 236.67$ y un valor $p = 3.17 \times 10^{-7}$, el cual es mucho menor que el nivel de significancia de 0.05. Por lo tanto, se rechaza la hipótesis nula (H_0) y se concluye que existe una relación lineal estadísticamente significativa entre el número de unidades producidas y los gastos generales. En consecuencia, el modelo de regresión lineal es adecuado para explicar y predecir los gastos generales a partir de las unidades producidas.

✓ Problema 4

Las ventas de línea blanca varían según el estado del mercado de casas nuevas: cuando las ventas de casas nuevas son buenas, también lo son las de lavaplatos, lavadoras de ropa, secadoras y refrigeradores. Una asociación de comercio compiló los siguientes datos históricos (en miles de unidades) de las ventas de línea blanca y la construcción de casas.

Construcción de casas (miles)	Ventas de línea blanca (miles)
2.0	5.0

Construcción de casas (miles)	Ventas de línea blanca (miles)
2.5	5.5
3.2	6.0
3.6	7.0
3.7	7.2
4.0	7.7
4.2	8.4
4.6	9.0
4.8	9.7
5.0	10.0

```
df=pd.DataFrame({
    'Construccion de casas':[2.0,2.5,3.2,3.6,3.7,4.0,4.2,4.6,4.8,5.0],
    'Ventas de linea blanca':[5.0,5.5,6.0,7.0,7.2,7.7,8.4,9.0,9.7,10.0]
})
df
```

	Construccion de casas	Ventas de linea blanca	
0	2.0	5.0	
1	2.5	5.5	
2	3.2	6.0	
3	3.6	7.0	
4	3.7	7.2	
5	4.0	7.7	
6	4.2	8.4	
7	4.6	9.0	
8	4.8	9.7	
9	5.0	10.0	

Próximos pasos: [Generar código con df](#) [New interactive sheet](#)

1.Establesca una variable dependiente (Y) y una variable independiente (X).

```
Y =df['Ventas de linea blanca']
X =df['Construccion de casas']
```

2.Realice un diagrama de dispersión para estos datos

```
# Importamos la librería matplotlib.pyplot y le damos el pseudónimo plt
import matplotlib.pyplot as plt

# Configuración general del gráfico
plt.figure(
    figsize=(6, 4),    # tamaño de la figura (ancho, alto) en pulgadas
    dpi=120            # resolución del gráfico
)

# Gráfico de dispersión
plt.scatter(
    X, Y,
    marker="o",        # forma: googlear "matplotlib.markers"
    color='#C7A97B',   # color de los puntos
    edgecolor='black', # borde de los puntos
    alpha=0.8,         # transparencia: 0 es invisible y 1 es color s
    s=50,              # tamaño de los puntos
)

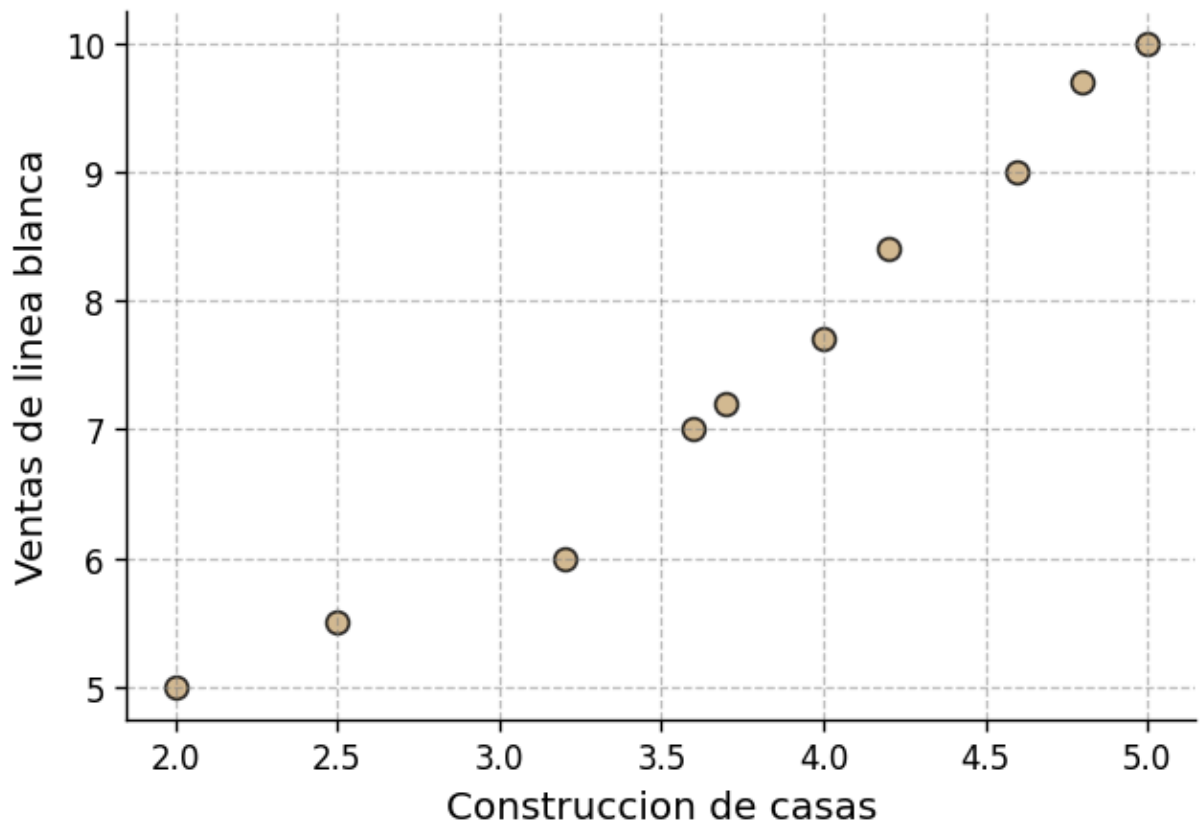
# Etiquetas de los ejes
# eje x
plt.xlabel(
    'Construccion de casas', # etiqueta del eje x
    fontsize=12 # tamaño de fuente
)

plt.ylabel(
    'Ventas de linea blanca', # etiqueta del eje y
    fontsize=12 # tamaño de fuente
)

# Fuente de los ticks
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)

# Márgenes
plt.gca().spines['right'].set_visible(False) # derecha
plt.gca().spines['top'].set_visible(False)   # superior

# Cuadrícula (opcional, pero didáctica)
plt.grid(
    visible=True,
    linestyle='--',
    linewidth=0.7,
    alpha=0.5,
    color="gray"
)
```



✓ 3.¿Los datos soportan la suposición de linealidad?

El gráfico de dispersión muestra una clara tendencia lineal. Se puede observar que conforme aumenta la construcción de casas (X), aumentan las ventas de línea blanca (Y). Con ello, se espera un coeficiente de correlación positivo y parece cumplirse el supuesto de linealidad.

✓ 4.Calcule el coeficiente de correlación e interprete los resultados

```
# Importar la función pearsonr desde scipy.stats
from scipy.stats import pearsonr

# Test de Pearson
# H0: rho = 0      (No hay correlación)
# H1: rho ≠ 0     (Sí hay correlación)
# alpha = 0.05

r, valor_p = pearsonr(X,Y)

print(f"Coeficiente de correlación: {r: 0.4f}")
print(f'valor_p: {valor_p: 0.4f}')
```

```
Coeficiente de correlación:  0.9808
valor_p:  0.0000
```

El coeficiente de correlación es $r = 0.9808$, lo que indica que existe una correlación lineal positiva muy fuerte entre las variables, ya que su valor es muy cercano a 1. Además, al ser positivo, significa que a medida que aumenta la variable X, también aumenta la variable Y. Por otra parte, el valor $p = 0.0000$ es menor que el nivel de significancia $\alpha = 0.05$, por lo que se rechaza la hipótesis nula (H_0) y se concluye que existe una correlación lineal estadísticamente significativa entre ambas variables.

✓ 5. Calcule el coeficiente de determinación e interprete los resultados

```
import statsmodels.api as sm
x_constante = sm.add_constant(X)
modelo = sm.OLS(Y, x_constante).fit()
y_calculada = modelo.predict(x_constante)
```

```
# Coeficiente de determinación
from sklearn.metrics import r2_score

r2 = r2_score(Y, y_calculada)

print(f"Coeficiente de determinación: {r2: 0.2%}")
```

```
Coeficiente de determinación:  96.19%
```

Con un coeficiente de determinación de:

$$R^2 = 0.9619$$

se indica que la Construcción de casas explica el 96.19% de la variabilidad en las Ventas de línea blanca. Por lo tanto, el modelo presenta una excelente capacidad de predicción,

ya que la mayor parte de la variación en las ventas de línea blanca puede explicarse a partir de la construcción de casas.

6. Obtenga la recta de regresión ajustada y gráfiquelo sobre el gráfico de dispersión

```
import statsmodels.api as sm
x_constante = sm.add_constant(X)
modelo=sm.OLS(Y,x_constante).fit()
y_calculada=modelo.predict(x_constante)

modelo.params
```

	0
const	1.016760
Construccion de casas	1.737564

dtype: float64

El modelo es:

$$\hat{y} = 1.016760 - 1.737564x$$

donde x es el factor construcción de casas

7. Obtenga un intervalo de confianza del 95% para la pendiente de la recta de regresión ajustada (b_1)

```
#intervalos de confianza
modelo.conf_int(alpha=0.05)
```

	0	1
const	-0.074848	2.108368
Construccion de casas	1.455693	2.019435

```
# Importamos la librería matplotlib.pyplot y le damos el pseudónimo plt
import matplotlib.pyplot as plt

# Configuración general del gráfico
plt.figure()
```

```

    figsize=(6, 4),    # tamaño de la figura (ancho, alto) en pulgadas
    dpi=120             # resolución del gráfico
)

# Gráfico de dispersión
plt.scatter(
    X, Y,
    marker="*",        # forma: googlear "matplotlib.markers"
    color='blue',      # color de los puntos
    edgecolor='#E38DBF', # borde de los puntos
    alpha=0.9,         # transparencia: 0 es invisible y 1 es color sólido
    s=50,              # tamaño de los puntos
)

plt.plot(
    X, y_calculada,
    color="blue",      # color de la línea de regresión
    alpha=0.9,         # transparencia: 0 es invisible y 1 es color sólido
    label="Recta de regresión" # Etiqueta para la leyenda
)

# Las líneas siguientes se usan típicamente en gráficos de residuales, n
# Si quieres ver la línea de regresión con la etiqueta, usa plt.legend()
# plt.axhline(
#     y=0,             # Donde está la línea horizontal
#     label="Recta de regresión", # Etiqueta
#     linestyle = "--", # Estilo de línea
#     color = "black",  # color
#     linewidth=1.0    # ancho de línea
# )

# Etiquetas de los ejes
plt.xlabel(
    'Promedio exámenes cortos', # etiqueta del eje x
    fontsize=12 # tamaño de fuente
)

plt.ylabel(
    'Promedio final', # etiqueta del eje y (cambiado de 'Residuales' y '
    fontsize=12 # tamaño de fuente
)

# Fuente de los ticks
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)

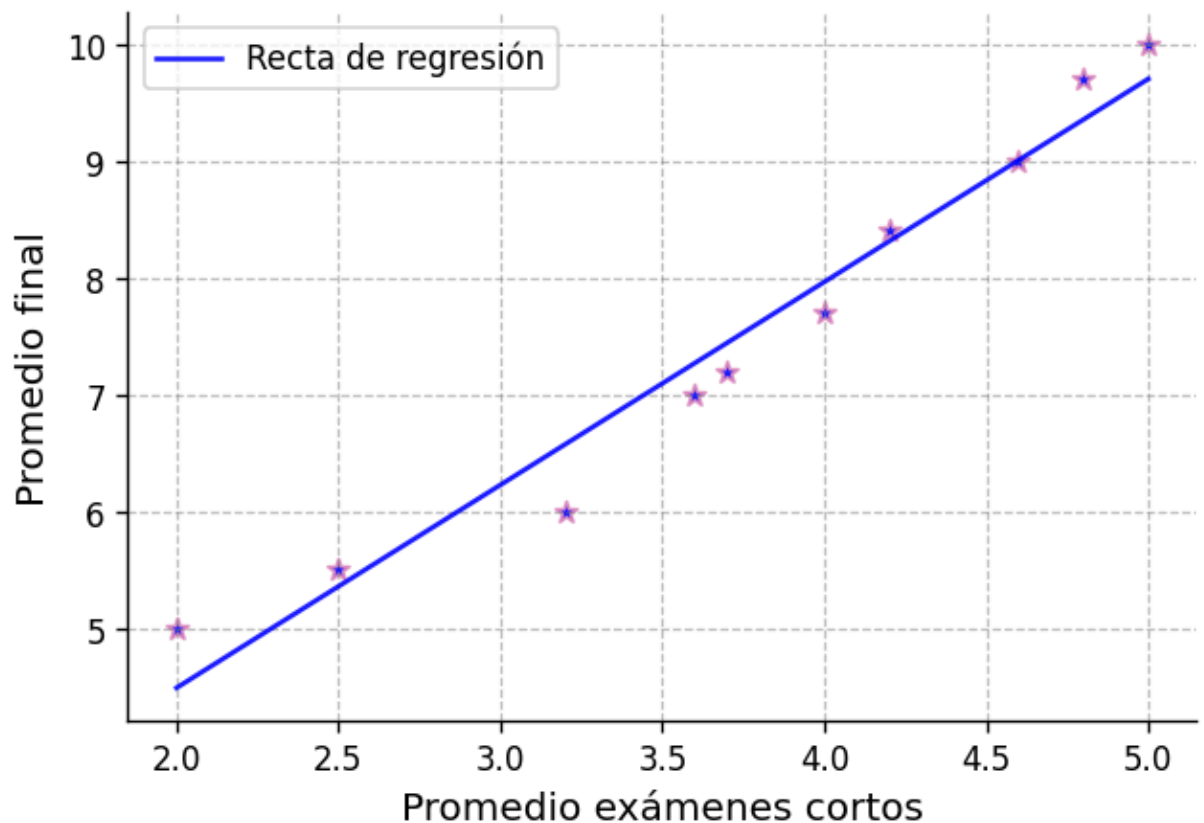
# Márgenes
plt.gca().spines['right'].set_visible(False) # derecha
plt.gca().spines['top'].set_visible(False)   # superior

# Cuadrícula (opcional, pero didáctica)

```

```
plt.grid(
    visible=True,
    linestyle='--',
    linewidth=0.7,
    alpha=0.5,
    color="gray"
)

plt.legend() # Muestra la leyenda con la etiqueta de la recta de regresión
plt.show() # Muestra el gráfico
```



8. Calcule los residuales y trace un nuevo gráfico de dispersión.
Comente, ¿Parece que se verifican los supuestos?

```
residuales = modelo.resid
```

```
# Importamos la librería matplotlib.pyplot y le damos el pseudónimo plt
import matplotlib.pyplot as plt

# Configuración general del gráfico
plt.figure(
    figsize=(6, 4), # tamaño de la figura (ancho, alto) en pulgadas
    dpi=120         # resolución del gráfico
)
```

```

)

# Gráfico de dispersión de residuales
plt.scatter(
    X, residuales,
    marker="o",          # forma: googlear "matplotlib.markers"
    color='#C28DE3',     # color de los puntos
    edgecolor='#DDA6FF',  # borde de los puntos
    alpha=0.9,           # transparencia: 0 es invisible y 1 es color s
    s=50,                # tamaño de los puntos
)

# Línea horizontal en y=0 para el gráfico de residuales
plt.axhline(
    y=0,
    color='black',
    linewidth=1.0,
    linestyle='--',
    label='y=0'
)

# Etiquetas de los ejes
# eje x
plt.xlabel(
    'Promedio exámenes cortos', # etiqueta del eje x
    fontsize=12 # tamaño de fuente
)

plt.ylabel(
    'Residuales', # etiqueta del eje y
    fontsize=12 # tamaño de fuente
)

# Leyenda
plt.legend(
    fontsize=10,
    loc='best',
    frameon=True
)

# Fuente de los ticks
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)

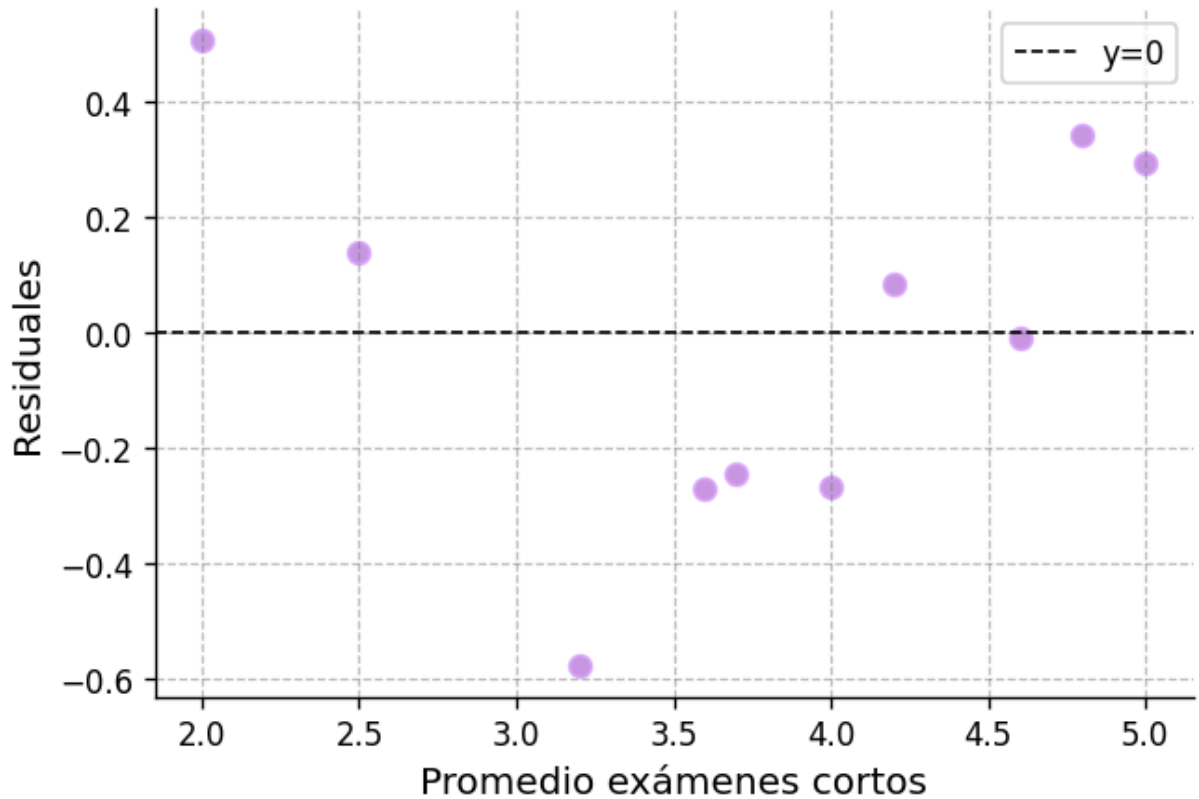
# Márgenes
plt.gca().spines['right'].set_visible(False) # derecha
plt.gca().spines['top'].set_visible(False)   # superior

# Cuadrícula (opcional, pero didáctica)
plt.grid(
    visible=True,

```

```
linestyle='--',
linewidth=0.7,
alpha=0.5,
color="gray"
)

plt.show() # Muestra el gráfico
```



Linealidad:

Se considera que este supuesto se cumple, ya que los residuos se distribuyen por encima y por debajo de la línea horizontal de cero, sin mostrar un patrón lineal o curvilíneo claramente definido.

Normalidad:

Este supuesto parece cumplirse de manera aproximada, debido a que la mayoría de los residuos se encuentra relativamente cercana a la línea de cero, aunque algunos presentan una mayor desviación.

Homoscedasticidad:

Se considera que este supuesto se satisface, ya que la dispersión vertical de los residuos es relativamente constante a lo largo de los valores del eje X y no se observa un cambio evidente en su variabilidad.

Independencia:

Se considera que este supuesto se cumple, debido a que los residuos no presentan una tendencia o secuencia sistemática, lo que sugiere que son independientes entre sí.

9. Realice la prueba de Shapiro para los residuales y comente el resultado.

```
from scipy.stats import shapiro
import matplotlib.pyplot as plt
import seaborn as sns

#test de shapiro
#H0: Hay normalidad
#H1: No hay normalidad

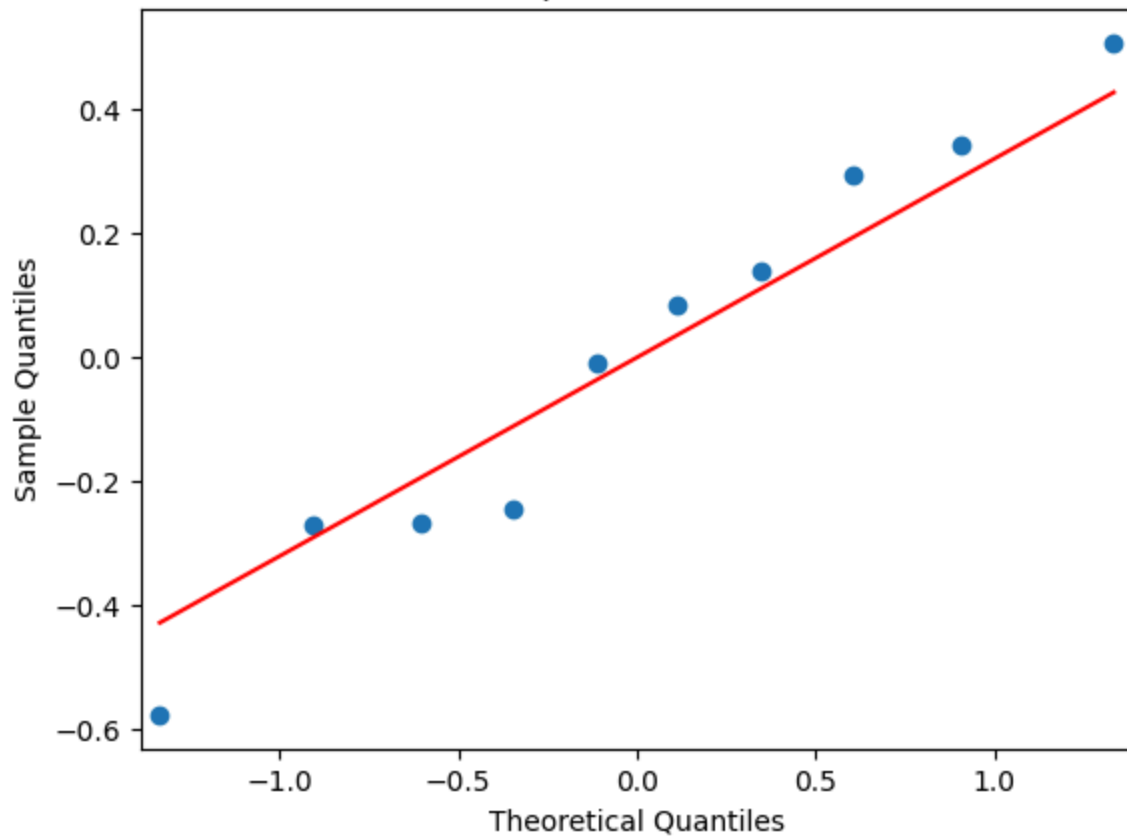
# Prueba de Shapiro-Wilk
stat, valor_p_sh = shapiro(residuales)
print(f"valor-p (Shapiro) = {valor_p_sh}")

# Visualización: Q-Q plot
sm.qqplot(residuales, line='s')
plt.title("Q-Q plot de residuos")
plt.show()

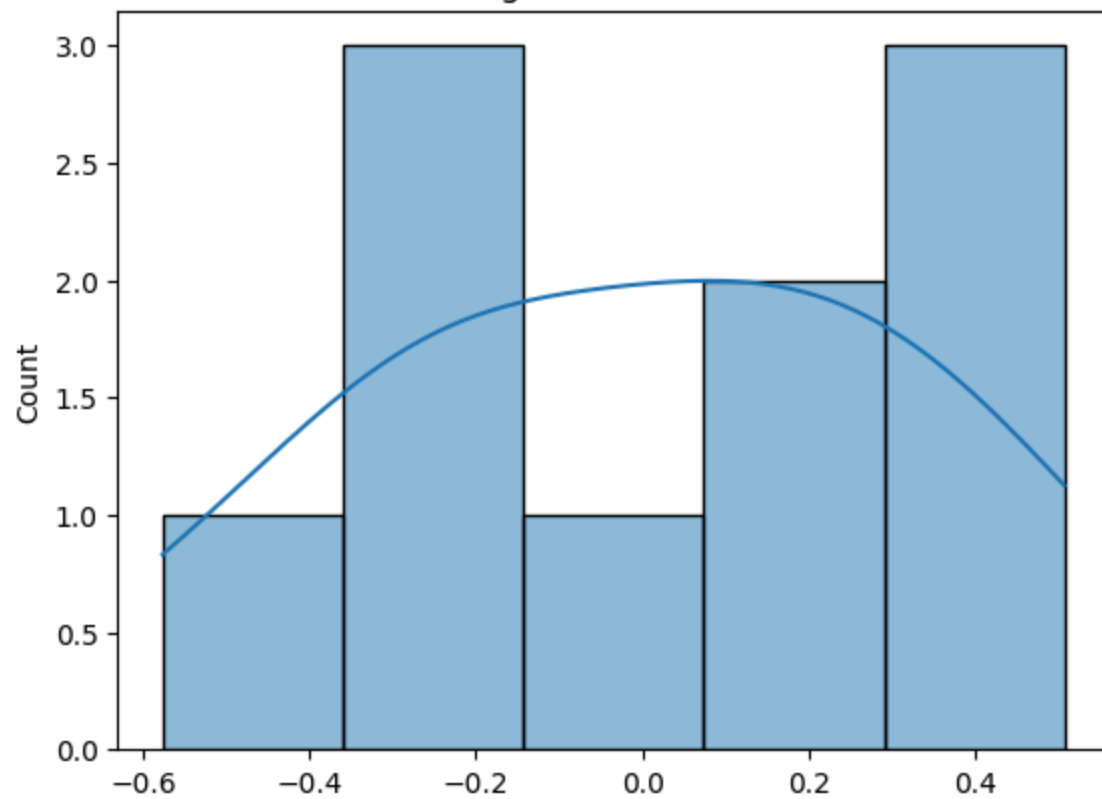
# Histograma
sns.histplot(residuales, kde=True)
plt.title("Histograma de residuos")
plt.show()
```

valor-p (Shapiro) = 0.8463507249054649

Q-Q plot de residuos



Histograma de residuos



- ✓ 10. Realice la prueba de Breusch-Pagan para los residuales y comente el resultado.

```
# Test de Breusch-Pagan
# H0: No hay heterocedasticidad (dispersión es igual en toda la recta)
# H1: Hay Heterocedasticidad (dispersión distinta a lo largo de la recta)
# alpha = 0.05

from statsmodels.stats.api import het_breuschpagan
import statsmodels.api as sm
x_constante = sm.add_constant(X)
_, valor_p_bp, _, _ = het_breuschpagan(residuales, x_constante)
print(f'valor-p de Breusch-Pagan: {valor_p_bp: 0.4f}\n')
```

```
valor-p de Breusch-Pagan: 0.1581
```

- ✓ 11. Utiliza la recta de regresión para interpolar dos valores y extrapolar uno. Comenta estos resultados.

```
# minimo casas: 2
# maximo casas: 5

# interpolar , valores de x entre 2 y 5

# interpolar 1
y_2 = modelo.predict([1, 2])
y_4 = modelo.predict([1, 4])

print(f"Para x = 2, y = {y_2[0]: 0.4f}")
print(f"Para x = 4, y = {y_4[0]: 0.4f}")
```

```
Para x = 2, y = 4.4919
Para x = 4, y = 7.9670
```

```
df["prediccion de ventas"] = y_calculada.round()
df
```

	Construccion de casas	Ventas de linea blanca	prediccion de ventas	 
0	2.0	5.0	4.0	
1	2.5	5.5	5.0	
2	3.2	6.0	7.0	
3	3.6	7.0	7.0	
4	3.7	7.2	7.0	
5	4.0	7.7	8.0	
6	4.2	8.4	8.0	
7	4.6	9.0	9.0	
8	4.8	9.7	9.0	
9	5.0	10.0	10.0	

Próximos pasos:
[Generar código con df](#)
[New interactive sheet](#)

```
# mínimo casas: 2
# máximo casas:5

# Extrapolar, valores de x que no estén entre 2 y 5

y_10 = modelo.predict([[1, 10]])

print(f"Para x = 10, y = {y_10[0]: 0.4f}")
```

Para x = 10, y = 18.3924

Interpolación :

Para $x = 2$, el valor estimado por la recta de regresión es $y = 4.4919$. Como este valor se encuentra dentro del rango de los datos observados, la predicción puede considerarse razonablemente confiable y consistente con el comportamiento mostrado por el modelo.

Para $x = 4$, el valor estimado por la recta de regresión es $y = 7.9670$. Al encontrarse también dentro del rango de los datos utilizados para construir el modelo, esta estimación es adecuada y representa una predicción válida.

Extrapolación :

Para $x = 10$, la recta de regresión proporciona una estimación de $y = 18.3924$, sin embargo, este valor de X se encuentra muy por encima del rango de los datos

observados (de 2 a 5). Por ello, el resultado debe interpretarse con precaución, ya que fuera del rango de estudio la relación entre las variables puede cambiar y el modelo lineal podría generar predicciones poco confiables.

✓ 12. Realice una tabla ANOVA e interprete el resultado.

```
import statsmodels.api as sm
from statsmodels.formula.api import ols
# Y ~ X
modelo_lineal = ols(
    '''Q('Ventas de linea blanca') ~ Q('Construccion de casas')''',
    data = df).fit()
tabla_anova = sm.stats.anova_lm(modelo_lineal)
tabla_anova
```

	df	sum_sq	mean_sq	F	PR(>F)	
Q('Construccion de casas')	1.0	25.976581	25.976581	202.069951	5.841003e-07	
Residual	8.0	1.028419	0.128552	NaN	NaN	

Próximos pasos:

[Generar código con tabla_anova](#)

[New interactive sheet](#)

En la tabla ANOVA se obtuvo un estadístico $F = 202.07$ y un valor $p = 5.84 \times 10^{-7}$, el cual es mucho menor que el nivel de significancia de 0.05. Por lo tanto, se rechaza la hipótesis nula (H_0) y se concluye que existe una relación lineal estadísticamente significativa entre la construcción de casas y las ventas de línea blanca. Esto indica que el modelo de regresión lineal es adecuado para explicar y predecir las ventas de línea blanca a partir de la cantidad de casas construidas.

✓ Problema 5

William C. Andrews, consultor de comportamiento organizacional de Victory Motorcycles, ha diseñado una prueba para mostrar a los supervisores de la compañía los peligros de sobrevigilar a sus trabajadores. Un trabajador de la línea de ensamble tiene a su cargo una serie de tareas complicadas. Durante el desempeño del trabajador, un inspector lo interrumpe constantemente para ayudarlo a terminar las tareas. El trabajador, después de terminar su trabajo, recibe una prueba psicológica diseñada para medir la hostilidad del trabajador hacia la autoridad (una alta puntuación implica una hostilidad baja). A ocho distintos trabajadores se les asignaron las tareas y luego se les

interrumpió para darles instrucciones útiles un número variable de veces (línea X). Sus calificaciones en la prueba de hostilidad se dan en el renglón Y.

número interrupciones al trabajador	5	10	10	15	15	20	20	25
calificación del trabajador en la prueba de hostilidad	58	41	45	27	26	12	16	3

```
import pandas as pd
df=pd.DataFrame({
    'Numero interrupciones':[5,10,10,15,15,20,20,25],
    'Calificación en prueba de hostilidad':[58,41,45,27,26,12,16,3]
})
df
```

	Numero interrupciones	Calificación en prueba de hostilidad
0	5	58
1	10	41
2	10	45
3	15	27
4	15	26
5	20	12
6	20	16
7	25	3

Próximos pasos: [Generar código con df](#) [New interactive sheet](#)

1.Establesca una variable dependiente (Y) y una variable independiente (X).

```
Y = df['Calificación en prueba de hostilidad']
X = df['Numero interrupciones']
```

2.Realice un diagrama de dispersión para estos datos

```
# Importamos la librería matplotlib.pyplot y le damos el pseudónimo plt
import matplotlib.pyplot as plt

# Configuración general del gráfico
plt.figure()
```

```

    figsize=(6, 4),    # tamaño de la figura (ancho, alto) en pulgadas
    dpi=120             # resolución del gráfico
)

# Gráfico de dispersión
plt.scatter(
    X, Y,
    marker="o",        # forma: googlear "matplotlib.markers"
    color='#C7A97B',   # color de los puntos
    edgecolor='black',  # borde de los puntos
    alpha=0.8,         # transparencia: 0 es invisible y 1 es color s
    s=50,              # tamaño de los puntos
)

# Etiquetas de los ejes
# eje x
plt.xlabel(
    'Ausencias', # etiqueta del eje x
    fontsize=12  # tamaño de fuente
)

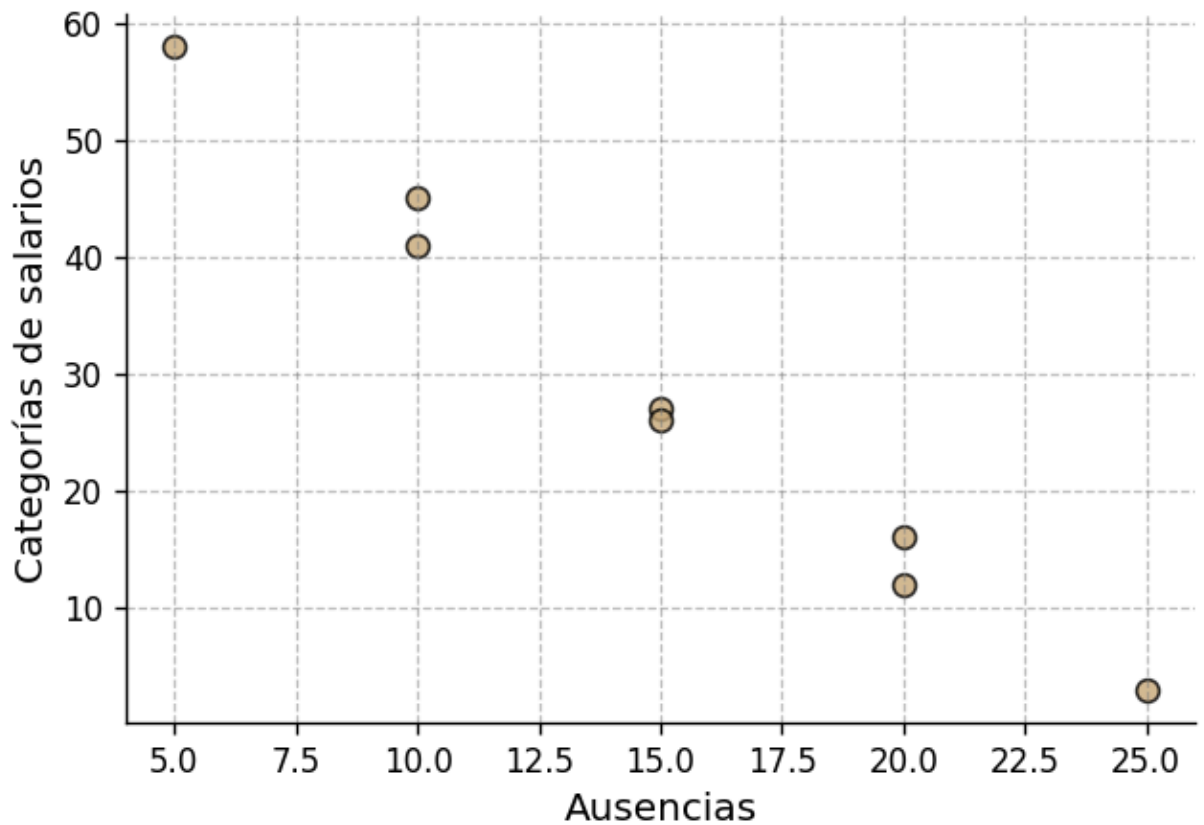
plt.ylabel(
    'Categorías de salarios', # etiqueta del eje y
    fontsize=12 # tamaño de fuente
)

# Fuente de los ticks
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)

# Márgenes
plt.gca().spines['right'].set_visible(False) # derecha
plt.gca().spines['top'].set_visible(False)   # superior

# Cuadrícula (opcional, pero didáctica)
plt.grid(
    visible=True,
    linestyle='--',
    linewidth=0.7,
    alpha=0.5,
    color="gray"
)

```



✓ 3.¿Los datos soportan la suposición de linealidad?

El gráfico de dispersión muestra una clara tendencia lineal. Se puede observar que conforme aumentan las ausencias (X), disminuyen las categorías de salarios (Y). Con ello, se espera un coeficiente de correlación negativo y parece cumplirse el supuesto de linealidad.

✓ 4.Calcule el coeficiente de correlación e interprete los resultados

```
# Importar la función pearsonr desde scipy.stats
from scipy.stats import pearsonr

# Test de Pearson
# H0: rho = 0      (No hay correlación)
# H1: rho ≠ 0      (Sí hay correlación)
# alpha = 0.05

r, valor_p = pearsonr(X,Y)

print(f"Coeficiente de correlación: {r: 0.4f}")
print(f'valor_p: {valor_p: 0.4f}')
```

```
Coeficiente de correlación: -0.9928
valor_p: 0.0000
```

El coeficiente de correlación es $r = -0.9928$, lo que indica que existe una correlación lineal negativa muy fuerte entre las variables, ya que su valor es muy cercano a -1. Además, al ser negativo, significa que a medida que aumenta la variable X, la variable Y disminuye. Por otra parte, el valor $p = 0.0000$ es menor que el nivel de significancia $\alpha = 0.05$, por lo que se rechaza la hipótesis nula (H_0) y se concluye que existe una correlación lineal negativa estadísticamente significativa entre ambas variables.

✓ 5. Calcule el coeficiente de determinación e interprete los resultados

```
import statsmodels.api as sm
x_constante = sm.add_constant(X)
modelo = sm.OLS(Y, x_constante).fit()
y_calculada = modelo.predict(x_constante)
```

```
# Coeficiente de determinación
from sklearn.metrics import r2_score

r2 = r2_score(Y, y_calculada)

print(f"Coeficiente de determinación: {r2: 0.2%}")
```

```
Coeficiente de determinación: 98.58%
```

Con un coeficiente de determinación de:

$$R^2 = 0.9858$$

se indica que el Número de interrupciones explica el 98.58% de la variabilidad en la Calificación en la prueba de hostilidad. Por lo tanto, el modelo presenta una excelente

capacidad de predicción, ya que prácticamente toda la variación en la calificación de la prueba de hostilidad puede explicarse a partir del número de interrupciones.

6. Obtenga la recta de regresión ajustada y gráfiquelo sobre el gráfico de dispersión

```
import statsmodels.api as sm
x_constante = sm.add_constant(X)
modelo=sm.OLS(Y,x_constante).fit()
y_calculada=modelo.predict(x_constante)

modelo.params
```

	0
const	70.5
Numero interrupciones	-2.8

dtype: float64

El modelo es:

$$\hat{y} = 70.5 - 2.8x$$

donde x es el número de interrupciones

7. Obtenga un intervalo de confianza del 95% para la pendiente de la recta de regresión ajustada (b_1)

```
#intervalos de confianza
modelo.conf_int(alpha=0.05)
```

	0	1
const	65.051384	75.948616
Numero interrupciones	-3.136296	-2.463704

```
# Importamos la librería matplotlib.pyplot y le damos el pseudónimo plt
import matplotlib.pyplot as plt

# Configuración general del gráfico
plt.figure()
```

```

figsize=(6, 4),    # tamaño de la figura (ancho, alto) en pulgadas
dpi=120            # resolución del gráfico
)

# Gráfico de dispersión
plt.scatter(
    X, Y,
    marker="*",      # forma: googlear "matplotlib.markers"
    color='blue',    # color de los puntos
    edgecolor='#E38DBF', # borde de los puntos
    alpha=0.9,       # transparencia: 0 es invisible y 1 es color sólido
    s=50,            # tamaño de los puntos
)

plt.plot(
    X, y_calculada,
    color="blue",    # color de la línea de regresión
    alpha=0.9,       # transparencia: 0 es invisible y 1 es color sólido
    label="Recta de regresión" # Etiqueta para la leyenda
)

# Las líneas siguientes se usan típicamente en gráficos de residuales, no
# Si quieres ver la línea de regresión con la etiqueta, usa plt.legend()
# plt.axhline(
#     y=0,          # Donde está la línea horizontal
#     label="Recta de regresión", # Etiqueta
#     linestyle = "--", # Estilo de línea
#     color = "black", # color
#     linewidth=1.0   # ancho de línea
# )

# Etiquetas de los ejes
plt.xlabel(
    'Promedio exámenes cortos', # etiqueta del eje x
    fontsize=12 # tamaño de fuente
)

plt.ylabel(
    'Promedio final', # etiqueta del eje y (cambiado de 'Residuales' y 'Residuos')
    fontsize=12 # tamaño de fuente
)

# Fuente de los ticks
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)

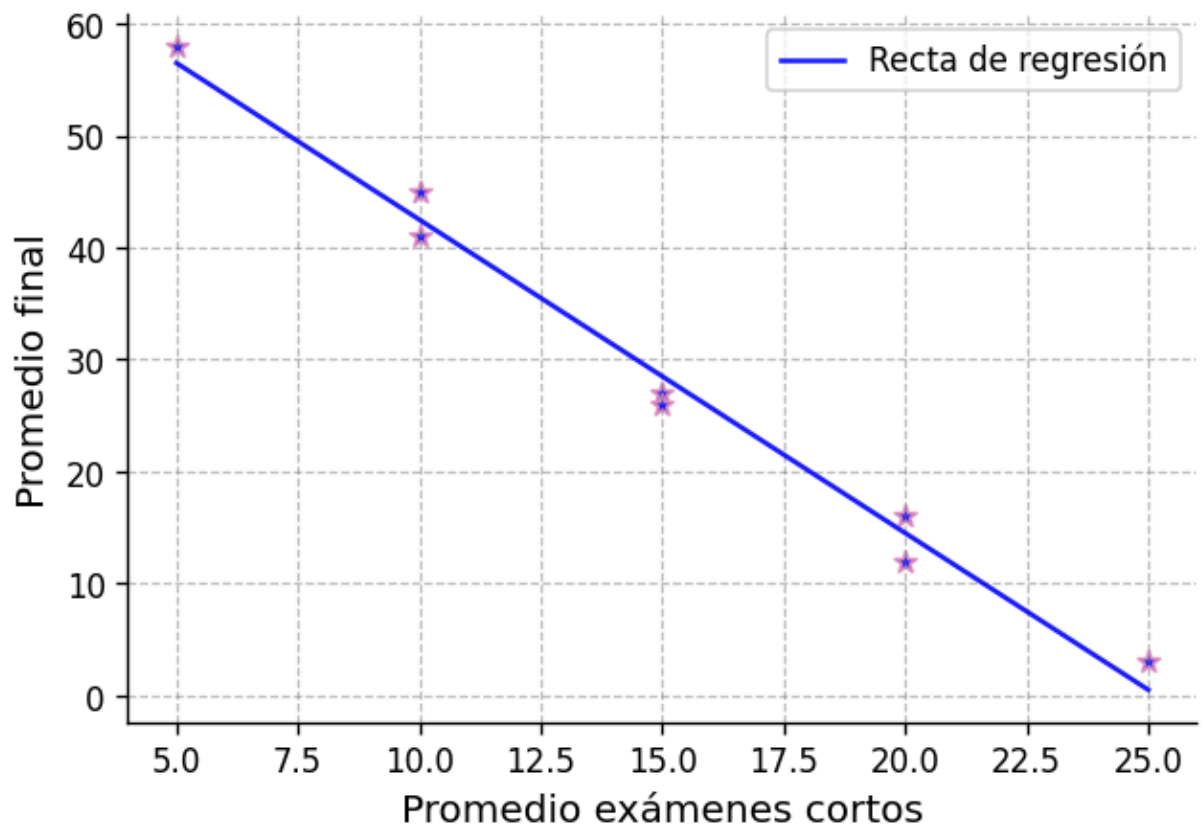
# Márgenes
plt.gca().spines['right'].set_visible(False) # derecha
plt.gca().spines['top'].set_visible(False)   # superior

# Cuadrícula (opcional, pero didáctica)

```

```
plt.grid(
    visible=True,
    linestyle='--',
    linewidth=0.7,
    alpha=0.5,
    color="gray"
)
```

```
plt.legend() # Muestra la leyenda con la etiqueta de la recta de regresión
plt.show() # Muestra el gráfico
```



8. Calcule los residuales y trace un nuevo gráfico de dispersión.

Comente, ¿Parece que se verifican los supuestos?

```
residuales = modelo.resid
```

```
# Importamos la librería matplotlib.pyplot y le damos el pseudónimo plt
import matplotlib.pyplot as plt
```

```
# Configuración general del gráfico
```

```
plt.figure(
    figsize=(6, 4), # tamaño de la figura (ancho, alto) en pulgadas
    dpi=120         # resolución del gráfico
)
```

```

)

# Gráfico de dispersión de residuales
plt.scatter(
    X, residuales,
    marker="o",          # forma: googlear "matplotlib.markers"
    color='#C28DE3',     # color de los puntos
    edgecolor='#DDA6FF',  # borde de los puntos
    alpha=0.9,           # transparencia: 0 es invisible y 1 es color s
    s=50,                # tamaño de los puntos
)

# Línea horizontal en y=0 para el gráfico de residuales
plt.axhline(
    y=0,
    color='black',
    linewidth=1.0,
    linestyle='--',
    label='y=0'
)

# Etiquetas de los ejes
# eje x
plt.xlabel(
    'Promedio exámenes cortos', # etiqueta del eje x
    fontsize=12 # tamaño de fuente
)

plt.ylabel(
    'Residuales', # etiqueta del eje y
    fontsize=12 # tamaño de fuente
)

# Leyenda
plt.legend(
    fontsize=10,
    loc='best',
    frameon=True
)

# Fuente de los ticks
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)

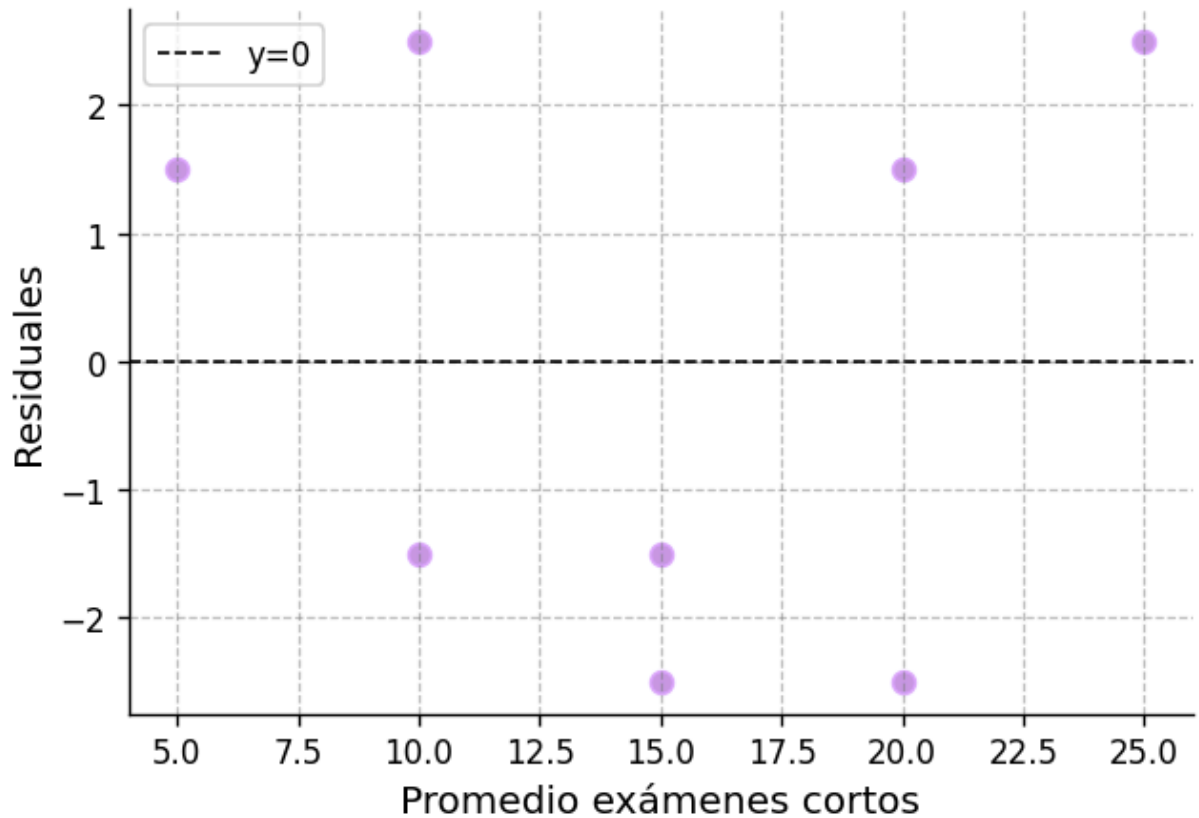
# Márgenes
plt.gca().spines['right'].set_visible(False) # derecha
plt.gca().spines['top'].set_visible(False)   # superior

# Cuadrícula (opcional, pero didáctica)
plt.grid(
    visible=True,

```

```
linestyle='--',
linewidth=0.7,
alpha=0.5,
color="gray"
)

plt.show() # Muestra el gráfico
```



Linealidad:

Se considera que este supuesto se cumple, ya que los residuos se distribuyen tanto por encima como por debajo de la línea horizontal de cero, sin mostrar una tendencia definida.

Normalidad:

Este supuesto parece no cumplirse, debido a que los residuos se encuentran distribuidos alrededor de la línea de cero, aunque algunos presentan una mayor separación.

Homoscedasticidad:

Se considera que este supuesto se satisface, ya que la dispersión vertical de los residuos se mantiene relativamente constante a lo largo de los valores del eje X.

Independencia:

Independencia:

Se considera que este supuesto se cumple, debido a que los residuos no presentan un patrón sistemático ni una secuencia evidente, lo que sugiere que son independientes entre sí.

9. Realice la prueba de Shapiro para los residuales y comente el resultado.

```
from scipy.stats import shapiro
import matplotlib.pyplot as plt
import seaborn as sns

#test de shapiro
#H0: Hay normalidad
#H1: No hay normalidad

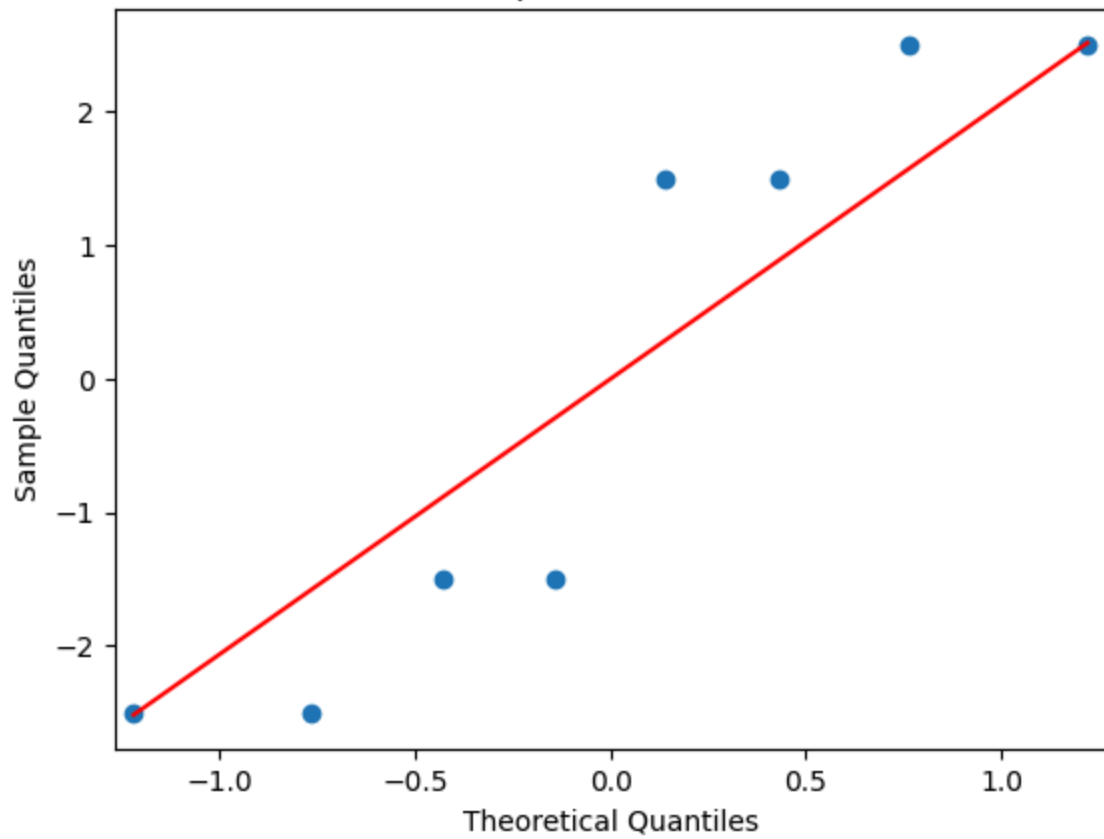
# Prueba de Shapiro-Wilk
stat, valor_p_sh = shapiro(residuales)
print(f"valor-p (Shapiro) = {valor_p_sh}")

# Visualización: Q-Q plot
sm.qqplot(residuales, line='s')
plt.title("Q-Q plot de residuos")
plt.show()

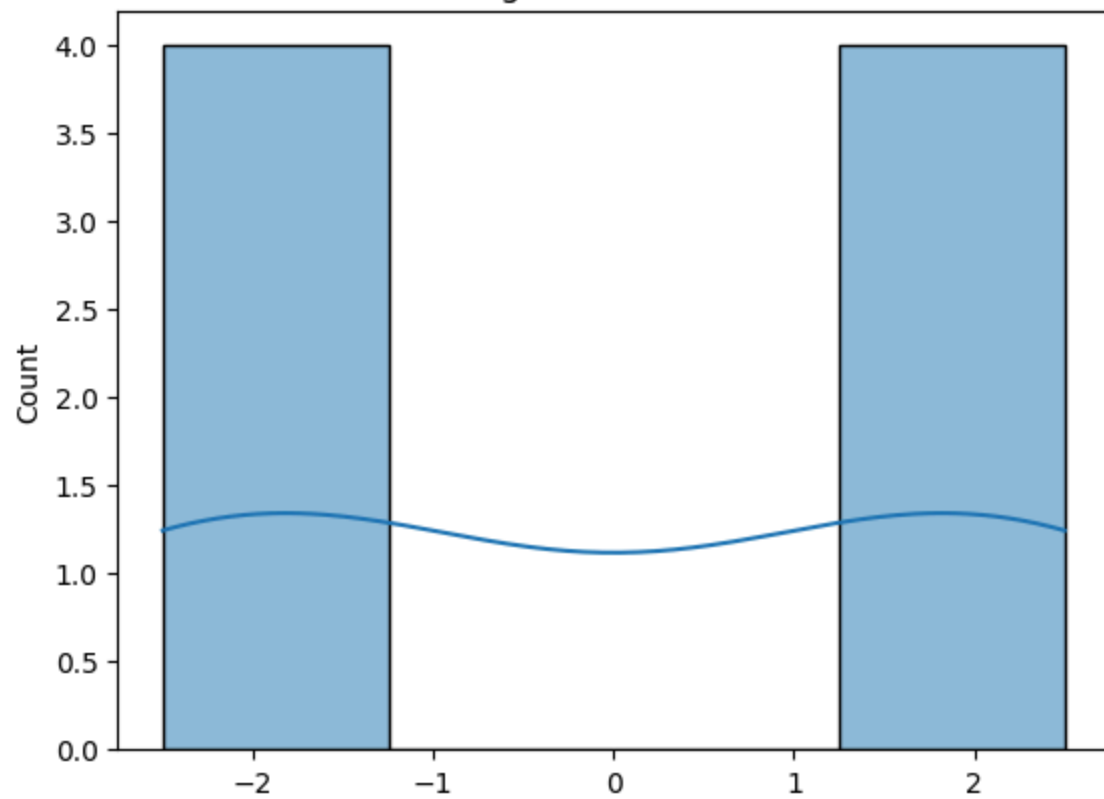
# Histograma
sns.histplot(residuales, kde=True)
plt.title("Histograma de residuos")
plt.show()
```

valor-p (Shapiro) = 0.054816491127669634

Q-Q plot de residuos



Histograma de residuos



- ✓ 10. Realice la prueba de Breusch-Pagan para los residuales y comente el resultado.

```
# Test de Breusch-Pagan
# H0: No hay heterocedasticidad (dispersión es igual en toda la recta)
# H1: Hay Heterocedasticidad (dispersión distinta a lo largo de la recta)
# alpha = 0.05

from statsmodels.stats.api import het_breuschpagan
import statsmodels.api as sm
x_constante = sm.add_constant(X)
residuales = modelo.resid # Recalculate residuals for the current problem
_, valor_p_bp, _, _ = het_breuschpagan(residuales, x_constante)
print(f'valor-p de Breusch-Pagan: {valor_p_bp: 0.4f}\n')
```

valor-p de Breusch-Pagan: 0.2482

- ✓ 11. Utiliza la recta de regresión para interpolar dos valores y extrapolar uno. Comenta estos resultados.

```
# minimo interrupciones:5
# maximo interrupciones:25

# interpolar , valores de x entre 5 y 25

# interpolar 1
y_8 = modelo.predict([1, 8])
y_17 = modelo.predict([1, 17])

print(f"Para x = 8, y = {y_8[0]: 0.4f}")
print(f"Para x = 17, y = {y_17[0]: 0.4f}")
```

Para x = 8, y = 48.1000
Para x = 17, y = 22.9000

```
df["calificacion predicha"] = y_calculada.round()
df
```

	Numero interrupciones	Calificación en prueba de hostilidad	calificacion predicha
0	5	58	56.0
1	10	41	42.0
2	10	45	42.0
3	15	27	28.0
4	15	26	28.0
5	20	12	15.0
6	20	16	15.0
7	25	3	1.0

```
# mínimo interrupciones: 5
# máximo interrupciones:25

# Extrapolar, valores de x que no estén entre 5 y 25

y_27 = modelo.predict([[1, 27]])

print(f"Para x = 27, y = {y_27[0]: 0.4f}")
```

Para x = 27, y = -5.1000

Interpolación :

Para $x = 8$ interrupciones, la recta de regresión estima una calificación de hostilidad de $y = 48.10$. Asimismo, para $x=17$ interrupciones, la calificación estimada es $y = 22.90$. Como ambos valores de X se encuentran dentro del rango observado (5 a 25 interrupciones), estas predicciones corresponden a una interpolación y pueden considerarse razonablemente confiables.

Extrapolación:

Para $x = 27$ interrupciones, la recta de regresión estima una calificación de $y = -5.10$. Sin embargo, este valor de X se encuentra fuera del rango de los datos utilizados para construir el modelo, por lo que la predicción debe interpretarse con precaución. El hecho de obtener una calificación negativa evidencia que las extrapolaciones pueden generar resultados poco realistas y menos confiables que las interpolaciones.

- ✓ 12.Realice una tabla ANOVA e interprete el resultado.

```

import statsmodels.api as sm
from statsmodels.formula.api import ols
# Y ~ X
modelo_lineal = ols(
    '''Q('Calificación en prueba de hostilidad') ~ Q('Numero interrupciones')
    data = df).fit()
tabla_anova = sm.stats.anova_lm(modelo_lineal)
tabla_anova

```

	df	sum_sq	mean_sq	F	PR(>F)
Q('Numero interrupciones')	1.0	2352.0	2352.000000	415.058824	9.090964e-07
Residual	6.0	34.0	5.666667	NaN	NaN

En la tabla ANOVA se obtuvo un estadístico $F = 415.06$ y un valor $p = 9.09 \times 10^{-7}$, el cual es mucho menor que el nivel de significancia de 0.05. Por lo tanto, se rechaza la hipótesis nula (H_0) y se concluye que existe una relación lineal estadísticamente significativa entre el número de interrupciones al trabajador y la calificación en la prueba de hostilidad. En consecuencia, el modelo de regresión lineal es adecuado para explicar y predecir la calificación de hostilidad a partir del número de interrupciones.